基于线性规划与滚动优化的微网购电策略优化研究

摘　要

本文针对微网与外部电网的电力调控优化问题，综合考虑光伏发电、小区负载、储能设备和外网电价等因素，从确定性场景推进至含预测不确定性、多阶段滚动调整以及实时电价波动的场景，建立并线性规划、鲁棒优化和多阶段随机规划模型，解决了微网购电策略制定与储能充放电调度问题。

针对问题一，在每日电价、小区负载、光伏发电功率已知的条件下，以全天购电费用最小化为目标函数，以功率平衡约束、储能动态方程、储能容量上下限、充放电功率限制以及日末电量循环约束作为约束条件。本文采用 PuLP 调用 CBC 求解器，高效求解，得到最优的购电策略。结果表明，在最优策略下全天购电费为**35126.95 元**，全天购电量为 **59482.699 kWh**。

针对问题二，在考虑小区负载和光伏存在不确定性的实际场景，本文构建了**“因果预测—鲁棒化修正—日前优化—实时模拟”**四阶段求解框架。设计**上周同期法**和**同季度同星期均值法**两种方法来预测小区负载，以及**前日同期法**和**近 7 天加权平均法**来预测光伏发电功率，并通过两两组合构成 4 种预测组合。通过全年 11 个月的滚动回测，得出上周同期与 7 天加权总费用最低，为 **1493.35 万元**，其中计划费 1463.52 万元，紧急费为 29.84 万元，单位购电成本 0.8324 元/kWh。

针对问题三，在日前计划基础上再引入 6:00、12:00、18:00 三个预报更新时刻，建立多阶段滚动优化模型。通过**安全分位数 q** 的敏感性分析，确定 q=0.95 为最优参数，此时总费用 **1447.76 万元**，比问题二下降了约 **3.05%**，其中计划费降低 168.73 万元，调整费 69.94 万元，紧急费增加 53.19 万元。调整购电费用主要集中在 14:00–18:00 时段，且调整量较大，因此分析出应该在 15:00 增加一次预报。

针对问题四，在外网电价实时波动条件下重新求解问题二和问题三。将目标函数中的固定电价替换为附件 4 中提供的逐日逐时段电价序列。结果表明：问题 4-3 总费用 1512.22 万元，较问题 4-2 的 1559.31 万元降低 47.09 万元（**降幅 3.0%**），单位购电成本由 0.8692 元/kWh 降至 **0.8429 元/kWh**。作为对比，固定电价场景下问题 3 较问题 2 降低 45.59 万元，降幅 3.05%。两种电价场景下滚动调整策略的降幅基本相当，说明多阶段滚动调整机制对于固定电价和实时波动电价均具有良好的适用性，验证了模型的适应性与稳健性。

本文的亮点在于：①构建了因果预测与数据驱动鲁棒化相结合的日前调度框架，安全余量由各方法自身的历史误差分位数自适应确定；②建立了多阶段滚动调整模型，通过可控风险增量换取大规模日前成本压降；③系统对比了固定电价与实时电价下的调度行为差异，量化了储能套利收益。

关键词：微网调度　线性规划　滚动优化　鲁棒修正　储能套利

一、 问题背景与重述

1.1 问题背景

为响应国家电网高质量发展和清洁能源体系建立等国家政策[1]，各地新兴微网系统用于解决光伏发电等分布式能源的本地存储、外部电网并网等问题。眼下，通过对小区负载、光伏发电量和储能设备的当前储备量等数据的收集整理，策略性在外网电价较低或分布式能源余点时间段为储能设备充电，并在外网高价时放电。由此，微网与外网的综合调控策略可以实现微网精确、经济购电和储能设备充放电量的合理管理。

1.2 问题重述

问题一：为满足小区用电需求，微网可提供电量不可低于小区负载，且为保持系统长期稳定及简化模型，须保证储能设备在 0：00 和 24：00 储电量相同。在每日电价、小区负载恒定，光伏发电的日功率预测数据已知的情况下，要求本模型根据以上已知条件做出具体购电策略，以达到供能充分和经济实惠的平衡。

问题二：在日常生活中，小区负载和光伏发电功率存在波动。若出现微网供电不足的情况，则需要以时价 5 倍的价格向外网紧急购电，其余时间购电费用以正常价格计算，在每天电价相同的前提下，要求本模型于每日 0：00 制定当日微网购电计划，尽可能避免紧急购电的情形，并平衡采购价格与供电负荷。

问题三：每日有四个时间点可对未来一日整点的光伏发电功率进行预报，并由此制定当天的购电计划。若实际用电情况与计划情况存在出入，记购电计划提出的购电量为计划购电量（以下简称“计划”），记实际用电量为调整购电量（以下简称“调整”），计划超出调整的部分以交易时刻电价的 50%计算，调整超出计划的部分以交易时刻电价的 1.5 倍计算。在此要求下，本模型需要综合电价、负载等因素，制定购电策略，使得计划费用、紧急补充费用、调整费用等费用的总和最经济。

问题四：在现实背景下，除了小区负载与光伏发电功率的波动之外，外网电价也会随时间变化，由此本模型需要结合所有波动因素，建立精确经济的购电策略，在满足小区负载的前提下，尽可能减少购电成本。

二、 问题分析

2.1 对问题一的分析

本题的核心内容是能量调度优化。微网每天微网每天需要从外网购电来满足小区负载需求，同时可以利用光伏发电（免费能源）和储能设备（通过峰谷电价差套

利）来降低购电成本。

由此，本文对问题一的解决方案做以下四个因素的分析：

光伏发电：白天光伏发电功率较高（峰值可达 7612 kW），可以部分或全部满足负载需求，多余电量可用于给储能充电。夜间光伏出力为零，需要从外网购电或释放储能。

电价差异：电价在一天中波动明显，峰时（8:00-22:00）均价约 0.93 元/kWh，谷时（22:00-次日 8:00）均价约 0.61 元/kWh，价差显著。储能设备可以在谷时低价充电、峰时高价放电以减少购电，从而节省费用。

储能约束：储能容量为 12000 kWh，充放电功率上限 5000 kW，电量需在[1200, 10800] kWh 范围内，且 0:00 和 24:00 储电量相同，从而保证模型的循环运行和稳定性。

优化本质：这是一个典型的线性规划问题，决策变量为每个时间段的购电量、充电量和放电量，目标函数为购电费用最小化，约束条件包括功率平衡、储能动态和容量限制。

2.2 对问题二的分析

本题的核心内容是基于历史数据预测 2025 年 2 月 1 日至 12 月 31 日的小区负载和光伏功率，并依此制定相应的购电计划。由于购电计划需要提前一天制定，而在具体执行过程中，预测存在的必然偏差会导致出现计划购电量不足以满足实际的用电量，这部分超出范围的电量将只能以交易时刻 5 倍价格紧急购入，造成成本显著增加。

由此，本文对问题二的解决方案提出以下四步求解法



因果预测：预测第 d 天时只允许使用 d 之前的历史数据，严格模拟 “提前一天” 的实际场景；

鲁棒性修正[4]：由各预测方法自身的历史误差分位数确定逐时段安全余量，对预测值做单边安全修正；

日前优化：以计划购电成本最小为目标建立线性规划（LP）模型，求各时段购电与充放电功率；

实时模拟：用实际数据模拟计划执行，统计紧急购电量与总费用，据此评价并优选预测组合与风险参数。

2.3 对问题三的分析

问题三在问题二的基础上，允许微网在每天 0:00 制定计划后，于 6:00、12:00、18:00 根据最新的光伏发电功率预报对购电策略进行调整。调整会产生额外费用：若最终购电量高于原计划，超出部分按交易时刻电价的 1.5 倍计费；若最终购电量低于原计划，不足部分按交易时刻电价的 50% 计违约费。此外，若实际供电仍不足，则需紧急购电，电价为交易时刻电价的 5 倍。

因此，问题三的核心是建立一个多阶段滚动优化模型：0:00 基于全天预报制定计划购电量，6:00、12:00、18:00 基于更新后的预报和实际已执行的运行状态，重新优化剩余时段的购电量，最终以实际光伏和负载进行实时模拟，计算紧急购电量和总费用。

2.4 对问题四的分析

问题四要求在外网电价实时波动的条件下，重新求解问题二和问题三。与问题二、三相比，模型结构、约束条件、求解流程完全一致，唯一变化是目标函数和费用结算中的电价系数由常数 c 变为实时序列 c_t。

这一变化使优化问题的性质发生根本改变：

固定电价下，目标函数 $min\sum_t c\cdot P_t^{buy}\Delta t$ 可化简为 $c\cdot min\sum_t P_t^{buy}\Delta t$，优化器仅关心总购电量，不关心购电时段分布，储能仅起 “削峰填谷” 作用。

实时电价下，目标函数 $min\sum_t c_t\cdot P_t^{buy}\Delta t$ 不可化简，优化器必须同时决定 “买多少” 和 “何时买”，储能从 “搬运工” 升级为 “套利者”，低买高卖成为核心机制。

因此，问题四的建模重点在于说明电价序列化如何改变优化行为，并通过对比分析验证套利机制的有效性。问题 4-3 还需额外分析在实时电价下是否需要引入其他时刻的光伏预报来制定调整策略。

三、 模型假设

1. 问题二、三中小区负载与光伏出力的预测误差具有统计稳定性，可用历史误差的分位数构造逐时段安全余量；
2. 每个时间段内的功率视为恒定值（10 分钟间隔内的平均值）；
3. 微网不向外部电网反向送电，各时段购电量非负；

储能设备在全年运行期间不发生故障，不考虑电池老化对容量的影响；

4. 忽略输电损耗与购电执行的延迟效应。

四、 符号说明

符号	单位	说明
P_t^{buy}	kW	时段 t 外网购电功率
P_t^{ch}	kW	时段 t 储能充电功率
P_t^{dis}	kW	时段 t 储能放电功率
P_t^{cur}	kW	时段 t 弃光功率
P_t^{plan}	kWh	时段 t 计划购电量
P_t^{eff}	kWh	时段 t 最终有效购电量
P_t^{emg}	kWh	时段 t 紧急购电量
$P_{\max}$	kW	充放电功率上限
L_t	kW	时段 t 小区负载功率
$\hat{L}_t^{(\mathrm{A})}$	kW	负载预测值（前日同期法）
$\hat{L}_t^{(\mathrm{B})}$	kW	负载预测值（7 日加权法）
G_t	kW	时段 t 光伏实际功率
$\hat{G}_t^{(\mathrm{A})}$	kW	光伏预测值（前日同期法）
$\hat{G}_t^{(\mathrm{B})}$	kW	光伏预测值（7 日加权法）
PV_t^{f}	kW	时段 t 光伏预报出力（问题三）
δ_{pv}	kW	光伏预测偏差（安全余量来源）
δ_{ld}	kW	负载预测偏差
Δ_t^{ld}	kW	负载预测安全余量
Δ_t^{pv}	kW	光伏预测安全余量
q	—	安全余量分位数
w_{m}	—	7 日加权法中各历史日权重
M	—	与预测日同季节、同星期属性的历史日集合
E_t	kWh	时段 t 末储能电量
$E_{\min}$ / $E_{\max}$	kWh	储能电量下限 / 上限
η	—	储能充放电效率
Δt	h	时段时长（10 min）

N/T	—	单日时段总数（N = T = 144）
τ	—	调整时刻（6:00 / 12:00 / 18:00）
p_t	元/kWh	时段 t 电价（问题一至三）
c_t	元/kWh	时段 t 实时电价（问题四）
A_t	kW	时段 t 实际可供电功率（购电+光伏）
D_t	kW	时段 t 实际负载需求功率
d_t^+	kWh	调整购电量高出计划的部分
d_t^-	kWh	调整购电量低于计划的部分
C_{plan} / C_{emg} / C_{adj} / C_{total}	元	计划 / 紧急 / 调整购电费用与全年总费用

五、 模型的建立与求解

5.1 问题一模型的建立与求解

5.1.1 模型建立

首先，对每个时间段 t（t=0,1,…,143）的决策变量进行命名，从外网购电的功率记为$P_{buy}(t) \geq 0$；储能充电功率记为$P_{ch}(t) \geq 0$；储能放电功率记为$P_{dis}(t) \geq 0$，以上各变量的单位均为千瓦（kW），由此建立目标函数，即全天购电费用最小化公式：

$$min\ C\ = \sum_{t=0}^{T-1} P_t^{\text{buy}}(t) \cdot \Delta t \tag{5.1}$$

其中dt = 1/6 小时，将功率转换为能量（kWh）后乘以电价得到费用（元）。

为满足微网稳定向小区供电，提供的电能始终不低于小区负载。同时为了模型的可重复性和长期稳定性，模型的运行需要满足以下约束条件：

(1)功率平衡约束

微网提供的总功率（购电+光伏+储能放电）必须不小于负载需求+储能充电：

$$P_t^{\text{buy}} + P_t^{\text{pv}} + P_t^{\text{dis}} \ \geq\ L_t + P_t^{\text{ch}}, \quad \forall t \tag{5.2}$$

该约束保证微网供电能力始终满足负载需求。

(2)储能动态方程

储能电量随时间的变化关系：

$$E_{t+1} = E_t + \eta \cdot P_t^{\text{ch}} \cdot \Delta t - \frac{P_t^{\text{dis}} \cdot \Delta t}{\eta}, \quad \forall t \tag{5.3}$$

其中充电效率$\eta = 0.9$表示充电时有能量损耗，放电效率$\eta = 0.9$表示放电时需要从储能中多取出能量。

(3)储能容量约束

$$E_{min} \le E(t) \le E_{max}, \quad \forall t$$
$$\text{即 } 1200 \le E(t) \le 10800 \text{ kWh} \tag{5.4}$$

(4)充放电功率约束

$$0 \le P_t^{\text{ch}} \le P_{\max} = 5000 \text{ kW} \tag{5.5}$$

$$0 \le P_t^{\text{dis}} \le P_{\max} = 5000 \text{ kW} \tag{5.6}$$

(5)边界条件

$$E(0) = E_0 = 6000\, kWh \tag{5.7}$$

$E(T) = E(0) = 6000\, kWh$（0:00 和 24:00 储电量相同）

综上所述，问题 1 的数学模型为如下线性规划问题：

$$\min \sum_{t=}^{T-1} pt \cdot P_t^{\text{buy}} \cdot \Delta t$$

s.t.

$$P_t^{\text{buy}} + P_t^{\text{pv}} + P_t^{\text{dis}} \ge L_t + P_t^{\text{ch}}, \quad \forall t$$

$$E_{t+1} = E_t + \eta \cdot P_t^{\text{ch}} \cdot \Delta t - \frac{P_t^{\text{dis}} \cdot \Delta t}{\eta}, \quad \forall t$$

$$1200 \le E(t) \le 10800, \quad \forall t$$

$$0 \le P_t^{\text{ch}} \le 5000, \quad \forall t$$

$$0 \le P_t^{\text{dis}} \le 5000, \quad \forall t$$

$$P_t^{\text{buy}} \ge 0, \quad \forall t$$

$$E(0) = 6000, \quad E(T) = 6000$$

该模型包含 432 个决策变量（144 个购电量+144 个充电量+144 个放电量）和约 720 个线性约束，可通过标准线性规划求解器（如 CBC、Gurobi 等）高效求解。

5.1.2 模型求解

针对此模型，本文提出采用 Python 的 PuLP 线性规划库[2]的方法求解上述问题。PuLP 是一个开源的线性规划建模工具，支持多种求解器后端。本文选用 CBC（Coin-or Branch and Cut）求解器[3]进行求解。步骤如下：求解流程如图 5.1 所示。

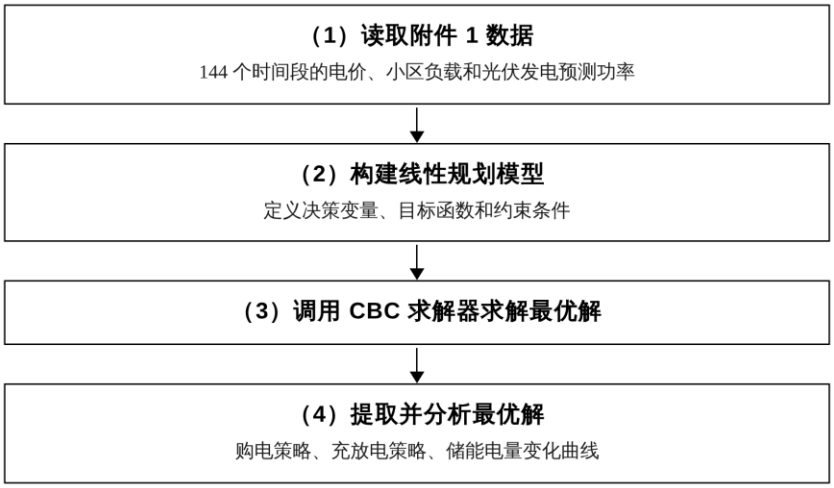


图 5.1 问题一求解流程图

在进入求解步骤之前，我们需要对附件 1 所提供的数据进行统计分析，如**图 5.2** 和**图 5.3** 所示

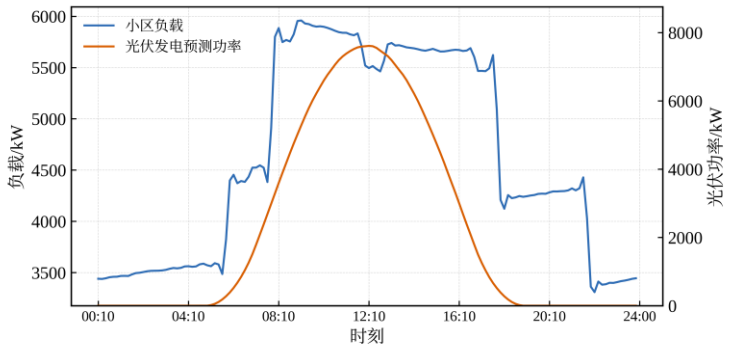


图 5.2 附件 1 数据统计特征

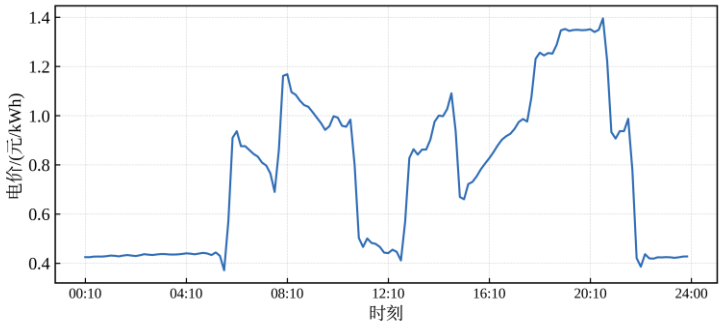


图 5.3 附件 1 典型日电价曲线

通过对图 5.2，图 5.3 与附件 1 数据的分析，结果如下：

指标	数值
电价特征	
峰时均价（8:00–22:00）	约 0.93 元/kWh
峰时最高价	1.3952 元/kWh
谷时均价（22:00–次日 8:00）	约 0.61 元/kWh
谷时最低价	0.3713 元/kWh
负载特征	
全天平均负载	约 4626 kW
峰值负载	5959 kW（约 09:10）
谷值负载	3309 kW
全天总用电量	约 111025 kWh
光伏特征	
有出力时段数	89 个（共 144 个）
光伏出力峰值	7612 kW
全天总发电量	约 55483 kWh
净负载为负的时段数	30 个

最终模型的求解结果如下：

在指定时间段的购电量（单位： kWh） 及全天的购电量（单位： kWh）和购电费（单位：元）见表 5.1 和表 5.2，储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量（单位： kWh）见表，完整的计划购电策略保存到附件 result1.xlsx 中

表 5.1 微网在指定时间段的购电量及全天的购电量和购电费

时间段	购电量	时间段	购电量	时间段	购电量
10:00-10:10	0	12:00-12:10	480.412	14:00-14:10	0
16:00-16:10	445.432	18:00-18:10	531.894	20:00-20:10	0
全天购电量		59482.699	全天购电费		35126.949

表 5.2 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量

时间段	充电量	放电量	时间段	充电量	放电量
0:00-4:00	4500	0	4:00-8:00	833.333	6365.841
8:00-12:00	4787.964	1702.997	12:00-16:00	5286.0352	91.101
16:00-20:00	0	5780.132	20:00-24:00	533.333	2859.868
0:00 储电量		6000	24:00 储电量		6000

5.2 问题二模型的建立与求解

5.2.1 模型建立

本题要求模型根据一定的负载与光伏预测，给出购电计划，对模型在此给出决策变量：$P_t^{\text{buy}}, P_t^{\text{ch}}, P_t^{\text{dis}}, P_t^{\text{cur}}, E_t,\ t = 1, \dots, N$，其中 P_t^{cur} 为弃光功率（松弛变量），变量总数 $5N = 720$。由此建立目标函数：

$$minC_{\text{plan}} = \sum_{t=1}^{N} p_t, P_t^{\text{buy}}, \Delta t \tag{5.8}$$

为满足小区负载，同时尽可能避免高价紧急购电导致的财产损失，模型的运行需要满足以下约束条件：

（1）功率平衡约束：

$$P_t^{\text{buy}} - P_t^{\text{ch}} + P_t^{\text{dis}} - P_t^{\text{cur}} = L_t^{\text{plan}} - G_t^{\text{plan}}, \qquad t = 1, \dots, N \tag{5.9}$$

（2）SOC 动态方程

将充电按效率折入，放 d 电按效率折出，则得到如下公式：

$$E_t = E_{t-1} + \eta, P_t^{\text{ch}}, \Delta t - \frac{P_t^{\text{dis}}, \Delta t}{\eta}, \qquad E_0 = E_{\text{init}} \tag{5.10}$$

其中 E_{init} 取前一天实际模拟结束时的电量，实现跨天连续。

（3）容量与功率约束：

$$E_{min} \le E_t \le E_{max}, \qquad 0 \le P_t^{\text{ch}} \le P_{max}, \qquad 0 \le P_t^{\text{dis}} \le P_{max} \tag{5.11}$$

（4）日末电量恢复约束：

为使电池每日保有约 4800 kWh 的应急缓冲（避免 LP 逐日榨干电池、夜间无电可放），要求：

$$E_N \geq E_{\text{start}} = 6000\text{kWh} \tag{5.12}$$

最后一天不施加该约束，允许自由用尽。验证表明该约束下每天 00:00 电量与前一天 24:00 严格衔接，且日末电量恰好回到 6000 kWh。

综上，该模型为标准稀疏线性规划，调用 linprog 全局求解，单日求解在秒级完成。

5.2.2 小区负载与光伏发电功率预测方法

为了得到更为准确经济的购电方案，基于已有数据的分析，可以对小区负载与光伏发电功率进行合理预测，用于指导购电方案制定。

首先对小区负载进行可视化分析见图 5.4、图 5.5

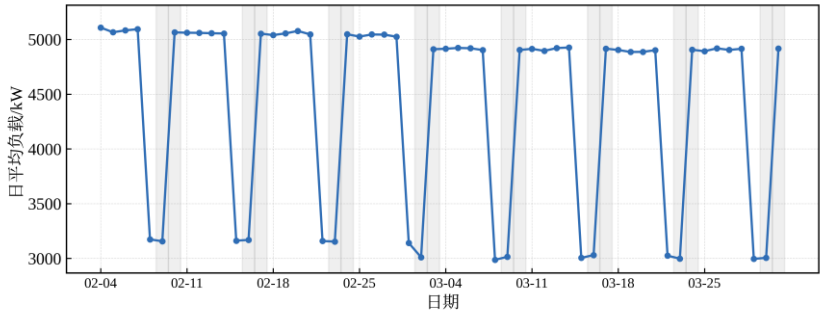


图 5.4 小区负载周期性特征

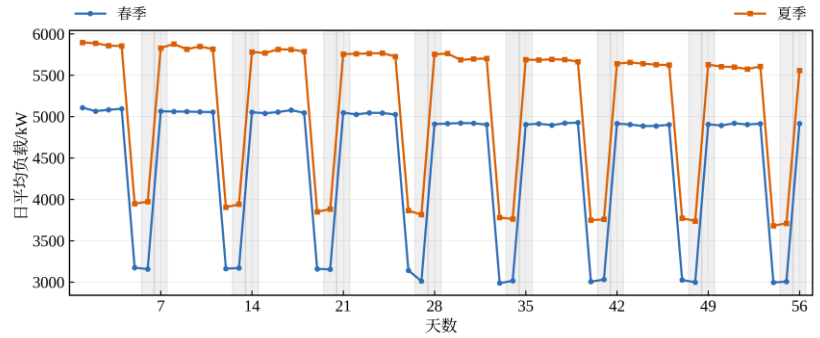


图 5.5 春夏两季小区平均日负载对比

从图 5.4 分析可得，小区负载在每 7 天有相似的循环，具有强周期性，从图 5.5 分析可得，在不同的季节里小区平均日用电量有较大的差异。因此，对于小区负载的预测，本文提出两种预测方案，“上周同期法”和“同季度同星期均值法”，说明如下：

上周同期法：负载具有强周期性（工作日 / 周末模式、日周期），取 7 天前同时段数据作为预测值：

$$\hat{L}_t^{(A)}(d) = L_t(d-7) \tag{5.13}$$

该法直接继承上一周期的实际水平，能自然捕捉星期类型差异，实现简单

同季度同星期均值法：同季节、同星期类型的负荷形态相近，取所有满足条件的历史日均值：

$$\hat{L}_t^{(B)}(d) = \frac{1}{|\mathcal{M}|}\sum_{k\in\mathcal{M}} L_t\,(k), \tag{5.14}$$

$$\mathcal{M} = \{k < d\colon \text{season}(k) = \text{season}(d),\ \text{weekday}(k) = \text{weekday}(d)\} \tag{5.15}$$

其中季节按春季（3–5 月）、夏季（6–8 月）、秋季（9–11 月）、冬季（12–2 月）划分。多日平均抑制了单日随机波动，且集合 $\mathcal{M}$ 仅含 d 之前的日期，满足因果性。

接着对光伏发电功率进行可视化分析见图 5.6，

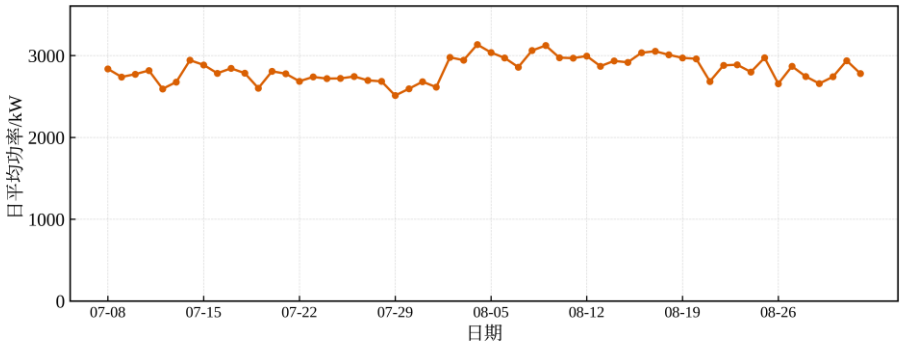


图 5.6 光伏出力特征

从图 5.6 分析可得，相邻的两日数据有较强的关联，且近七天的存在一定的趋势，因此本文提出如下两种解决方案，即“前日同期法”和“7 日加权平均法”，说明如下：

前日同期法：相邻两日天气具有持续性：

$$\hat{G}_t^{(A)}(d) = G_t(d-1) \tag{5.16}$$

近 7 天加权平均法： 按“越近权重越大”原则对近 7 天同时段数据加权：

$$\hat{G}_t^{(B)}(d) = \frac{\sum_{m=1}^{7} w_m, G_t(d-m)}{\sum_{m=1}^{7} w_m}, \tag{5.17}$$

$$w_m = 0.30,0.25,0.20,0.15,0.07,0.02,0.01$$

夜间（无光照时段）预测值强制置零。

针对这一思路，本文对小区负载和光伏发电功率的预测各设计的两种方案，经交叉组合最后可得 4 种组合预测方案，进行对比择优。

5.2.3 模型的求解

日前计划确定后，用实际负载与光伏数据逐时段模拟执行。记实际可用电力 $A_t = G_t^{\text{act}} + P_t^{\text{buy}}$，实际负荷 $D_t = L_t^{\text{act}}$：

$$P_t^{\text{ch}} = min\left(A_t - D_t, P_{max}, \frac{E_{max} - E_{t-1}}{\eta, \Delta t}\right) \tag{5.18}$$

$$P_t^{\text{dis}} = min\left(D_t - A_t, P_{max}, \frac{\eta, (E_{t-1} - E_{min})}{\Delta t}\right), P_t^{\text{emg}} = max\left(D_t - A_t - P_t^{\text{dis}}, 0\right) \tag{5.19}$$

$$C_{\text{plan}} = \sum_t p_t P_t^{\text{buy}} \Delta t\,, C_{\text{emg}} = \sum_t 5p_t, P_t^{\text{emg}}, \Delta t\,, C_{\text{total}} = C_{\text{plan}} + C_{\text{emg}} \tag{5.20}$$

5.2.4 结果与灵敏性分析

四种预测组合全年购电费用对比如下表所示

预测组合	计划购电费用/元	紧急购电费用/元	全年总费用/元
负载 A + 光伏 A	15 032 851	275 233	15 308 084
负载 A + 光伏 B	**14 635 150**	**298 350**	**14 933 500**
负载 B + 光伏 A	14 903 031	509 590	15 412 621
负载 B + 光伏 B	14 505 853	520 245	15 026 098

从表中结果看，最优组合为“负载 A（上周同期）+光伏 B（近 7 天加权）”，全年总费用 14,933,500 元，比次优组合低约 9.26 万元；四种组合的计划购电费用差异不大（极差约 52.77 万元），但紧急购电费用差距悬殊，负载 A 的两种组合紧急费仅 27.5 万~29.8 万元，而负载 B 的两种组合高达 51.0 万~52.0 万元，后者约为前者的 1.8 倍，主要原因是负载 A 完整保留了星期类型特征，预测值不会系统性偏低，而负载 B 采用同季度同星期均值，容易在高负载日低估实际需求，从而触发了 5 倍电价的紧急购电；光伏 B 通过近 7 天加权平均平滑了单日天气异常，比前日同期法更稳健，在相同负载方法下可降低总费用约 37 万~39 万元；因此，负载预测方法的优先级高于光伏预测方法，最优组合的本质是用上周同期法保证负载时序模式准确、用近 7 天加权法保证光伏统计稳健，两者结合实现全年总费用最小化。

指定日期的紧急购电结果见表 5.3

表 5.3 指定日期微网紧急购电结果

2025.3.20		2025.6.21		2025.9.23		2025.12.21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
0:00-24:00	0	0:00-24:00	0	9:00-9:50	256.486	7:50-8:00	48.915
				20:10-21:00	1886.274	10:30-10:50	315.737
				21:10-21:40	89.236	20:30-20:40	552.181

算法用于预测小区负载与光伏发电功率，需要对可能存在的预测系统误差进行因果估计与鲁棒化修正，具体操作如下：

$$\delta_t^{\mathrm{pv}}(k) = max(\hat{G}_t(k) - G_t(k),0), \delta_t^{\mathrm{ld}}(k) = max(L_t(k) - \hat{L}_t(k),0) \tag{5.21}$$

5.3 问题三模型的建立与求解

5.3.1 模型的建立

针对问题三所设情形，可以将其分为计划阶段与调整阶段进行分别建模：

计划阶段建模：

本文选取 0：00 时为当日计划购电时间，根据附件 3 中 0:00 发布的未来 24 小时光伏预测，制定全天 144 个时段的计划购电量。以全天购电费用最小为目标，公式如下：

$$min \sum_{t=1}^{T} c_t P_t^{\mathrm{buy}} \Delta t \tag{5.22}$$

问题三是问题二的扩展，所以其约束条件既有相同也有差异，为保证问题三模型的稳定运行，需要的约束条件如下：

（1） 功率平衡约束

$$P_t^{\mathrm{buy}} + P_t^{\mathrm{dis}} + PV_t^f = L_t + P_t^{\mathrm{ch}} + P_t^{\mathrm{cur}}, \forall t \tag{5.23}$$

（2） 储能电量动态约束

$$E_t = E_{t-1} + \eta P_t^{\mathrm{ch}} \Delta t - \frac{P_t^{\mathrm{dis}} \Delta t}{\eta}, \forall t \tag{5.24}$$

其中 E_0 为当天初始储能电量，由前一天 24:00 的储能状态传递而来。

（3） 储能电量上下限约束

$$E_{min} \le E_t \le E_{max}, \forall t \tag{5.25}$$

（4） 充放电约束

$$0 \le P_t^{\text{ch}} \le P_{max}, 0 \le P_t^{\text{dis}} \le P_{max}, \forall t \tag{5.26}$$

（5） 购电量与弃电量非负约束

$$P_t^{\text{buy}} \ge 0, P_t^{\text{cur}} \ge 0, \forall t \tag{5.27}$$

（6） 日末储能约束

仅在计算区间最后一天（2025 年 12 月 31 日）要求日末储能不低于 6000 kWh，其余日期不强制日末储能水平，只需满足上下限即可。即：

$$E_T \ge 6000\text{(仅最后一天)} \tag{5.28}$$

综上所述，本模型是一个线性规划问题，决策变量为 $P_t^{\text{buy}}, P_t^{\text{ch}}, P_t^{\text{dis}}, P_t^{\text{cur}}, E_t$，共 $5T$ 个变量。利用 MATLAB 的 linprog 函数可求得全天计划购电量 P_t^{plan}

调整阶段模型：

由于本文选取 0：00 作为计划购电时间点，则根据题意，在 6：00，12：00，18：00 三个时刻，需要根据最新预报和实际已执行时段的储能状态，对剩余时段的购电量进行调整。设调整时刻为 τ，剩余时段为 $t = \tau, \tau+1, \dots, T$，剩余时段数为 $M = T - \tau + 1$。

在调整阶段，本文首要目标就是最小化调整费用。定义最终购电量 P_t^{buy} 相对于原计划 P_t^{plan} 的正偏差和负偏差，即：

$$d_t^+ = max(P_t^{\text{buy}} - P_t^{\text{plan}}, 0), d_t^- = max(P_t^{\text{plan}} - P_t^{\text{buy}}, 0) \tag{5.29}$$

调整费用为：

$$min \sum_{t=\tau}^{T} [\,1.5 c_t d_t^+ + 0.5 c_t d_t^-] \Delta t \tag{5.30}$$

约束条件与计划阶段类似，但需增加差额定义约束，即：

$$P_t^{\text{buy}} - d_t^+ + d_t^- = P_t^{\text{plan}}, \forall t \ge \tau \tag{5.31}$$

同时，功率平衡、储能动态、上下限、功率限制等约束均作用于剩余时段。初始储能电量 $E_{\tau-1}$ 由实际已执行时段的模拟结果给出：在每次调整前，利用实际光伏和实际负载，按照已确定的购电策略模拟从 0:00 到 $\tau - 1$ 时段的运行，得到真实的储能电量。

调整阶段同样构建为线性规划模型，决策变量为$P_t^{\text{buy}}, P_t^{\text{ch}}, P_t^{\text{dis}}, P_t^{\text{cur}}, E_t, d_t^+, d_t^-$，共 $7M$ 个变量。求解后得到调整后的购电量，并更新最终有效购电量 P_t^{eff}。

5.3.2 模型的求解

在确定计划与调整算法后，单日模型的循环求解过程为：单日循环求解流程如图 5.7 所示。

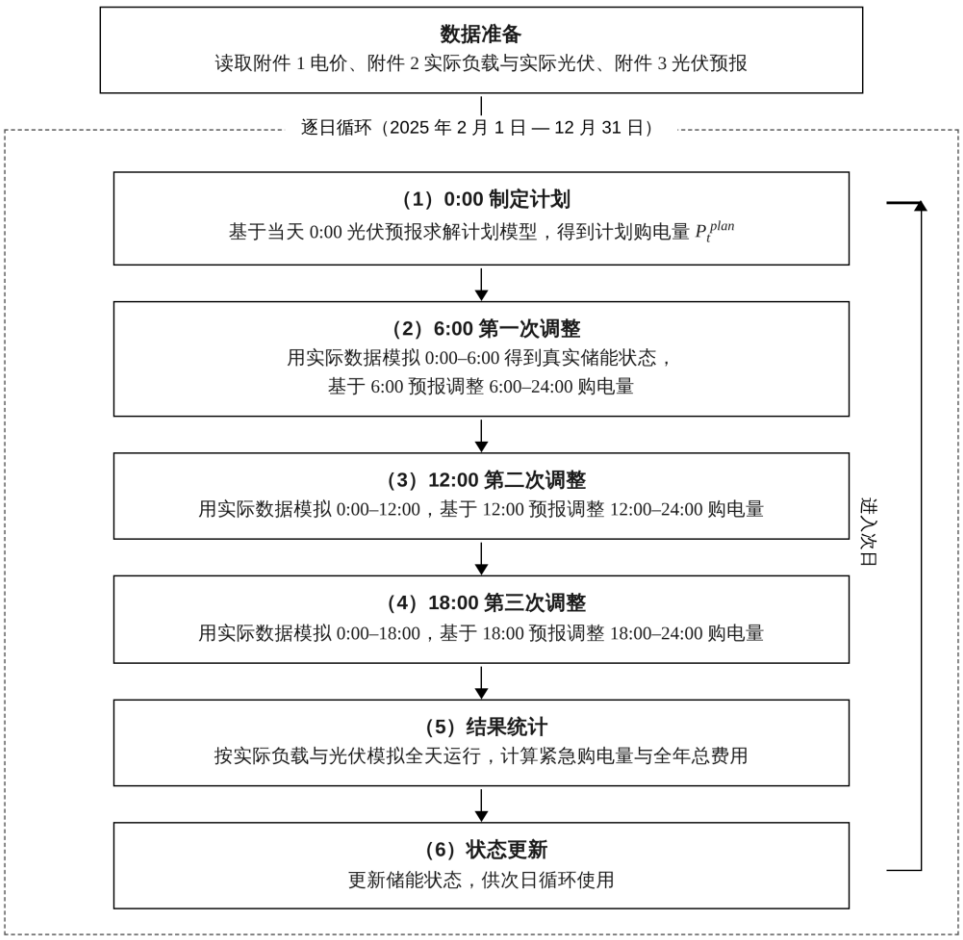


图 5.7 问题三求解流程图

5.3.3 结果分析

全年模拟结果统计如下表所示

统计项目	数值	单位
统计天数	334	天
启用调整购电策略天数	59（占 17.7%）	天
计划购电量总计	21 128 898	kWh
实际需求总量	17 939 189	kWh
计划与需求差额	3 189 709	kWh
紧急购电量总计	138 949（占 0.77%）	kWh
计划购电费用	12 947 876	元
调整购电费用	699 402	元

统计项目	数值	单位
全年总费用	14 477 588	元
光伏预报平均偏差	2.11	kW
负载预报平均偏差	174.81	kW

通过分析可得，问题三全年总费用为 14477588 元，较问题二的 14933500 元降低约 45.6 万元（降幅 3.05%），说明多阶段滚动调整机制在光伏预报存在偏差的条件下能够有效降低成本；全年 334 天中仅 59 天（17.7%）启用了调整策略，其余 275 天原计划更优，表明“调整 vs 不调整”对比机制运作合理，避免了无谓调整；计划购电量 2112.9 万 kWh 比实际需求 1793.9 万 kWh 多出 318.97 万 kWh，主要来源于负载预报本身偏保守（平均偏差+174.81 kW）以及夜间安全上浮，多购电量的代价是计划费偏高，但换来了紧急购电率仅 0.77%的低水平，紧急购电量 13.9 万 kWh 远低于问题二的 29.8 万 kWh；光伏预报经校正后平均偏差仅 2.11 kW，几乎消除系统性偏差，为计划阶段的准确性提供了保障；调整费仅 69.94 万元，占计划费的 5.4%，远低于其带来的紧急购电费节省，说明调整机制是经济理性的。

指定日期的结果见表 5.4～表 5.6

表 5.4 四个指定日期微网在指定时间段的购电量及全天的购电量和购电费

指定时间段	**2025.3.20**	**2025.6.21**	**2025.9.23**	**2025.12.21**
	购电量/kWh	**购电量/kWh**	**购电量/kWh**	**购电量/kWh**
10:00-10:10	0	0	0	0
12:00-12:10	499.9154	0	300.3883	941.1368
14:00-14:10	0	0	0	0
16:00-16:10	646.3392	169.3185	582.9239	871.6729
18:00-18:10	733.5257	0	826.0968	788.227
20:00-20:10	0	0	14.6689	0
全天购电量/kWh	69873.271	35994.237	71037.184	96419.1849
全天购电费/元	43292.6151	20622.1838	44909.8593	62440.9981

表 5.5 四个指定日期储能设备充放电量及 0:00 和 24:00 的储电量

时间段	**2025.3.20**		**2025.6.21**		**2025.9.23**		**2025.12.21**	
	充电量/kWh	放电量/kWh	充电量/kWh	放电量/kWh	充电量/kWh	放电量/kWh	充电量/kWh	放电量/kWh
0:00-4:00	5279.4217	4.5415	1216.1748	0	5206.993	0	4521.8558	56.8176

4:00-8:00	47.1674	5862.494	600.2221	3932.683	36.8931	6544.792	819.9038	1466.713
8:00-12:00	4625.5867	2777.377	8299.9885	0	5091.0988	1896.616	5440.9075	7816.33
12:00-16:00	6308.2259	254.7228	0	27.005	5963.0732	542.3547	7456.3088	2405.304
16:00-20:00	0	5105.328	153.4866	5463.581	8.0355	5304.597	768.0216	5185.732
20:00-24:00	4510.9454	2930.217	4881.5601	2488.938	5028.1837	2629.994	4834.0369	2695.283
0:00 储电量/kWh	6053.567		6064.901		6113.706		6759.302	
24:00 储电量/kWh	5931.468		6465.405		6516.386		6409.367	

表 5.6 四个指定日期微网紧急购电明细

日期	紧急购电时间段	紧急购电量/kWh
2025.3.20	无紧急购电	
2025.6.21	无紧急购电	
2025.9.23	无紧急购电	
2025.12.21	7:50-8:00	16.2401
	8:00-8:10	14.516
	10:40-10:50	33.6184

最终的完整结果保存在 result3.xlsx

5.3.4 模型灵敏性分析

问题三的鲁棒化策略中，安全分位数Q_{SAFE}是一个关键参数，直接影响安全余量的大小，进而影响计划购电费、调整费和紧急购电费。为确定最优，本文对Q_{SAFE}开展敏感性分析。敏感性分析结果如图 5.8 所示。$Q_{SAFE} \in 0.70,0.80,0.90,0.95,0.99$

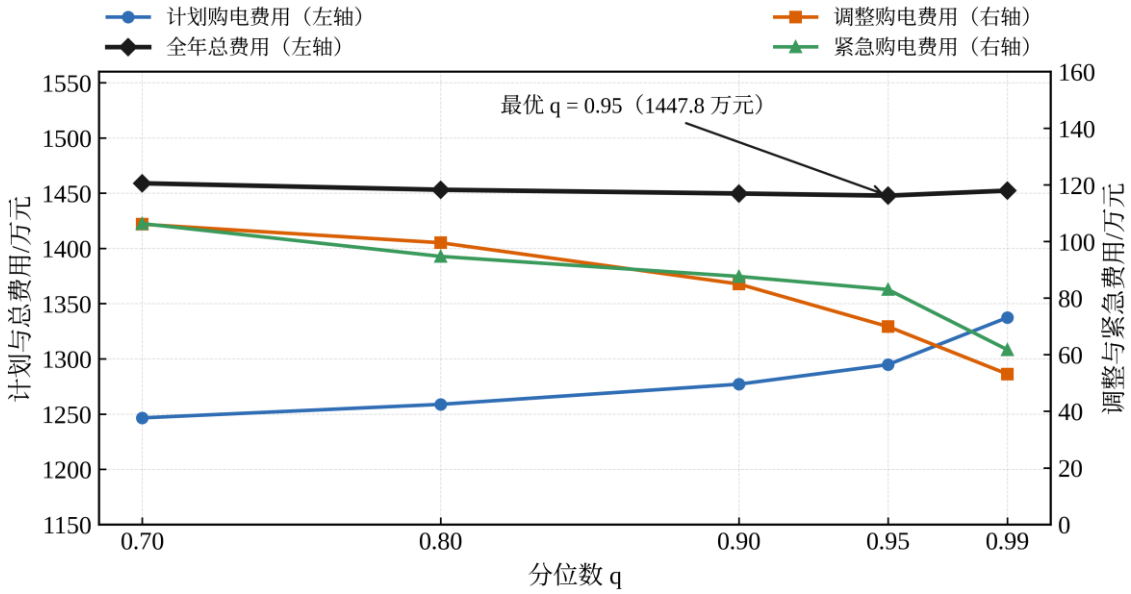


图 5.8 安全分位数 q 的敏感性分析

结合图 5.8 可得到以下分析

（1）**计划费随 Q_{SAFE} 上升**

随 Q_{SAFE} 增大，负载安全上浮量增大，计划购电量从 2051 万 kWh 增至 2162 万 kWh，计划购电费相应从 1246.56 万元升至 1337.38 万元，涨幅 7.3%。

（2）**调整费随 Q_{SAFE} 下降**

计划越保守，留给调整的空间越小，启用调整策略的天数从 99 天降至 44 天，调整费从 106.09 万元降至 53.17 万元，降幅 49.9%。

（3）**紧急费随 Q_{SAFE} 下降**

安全余量越大，紧急购电量从 18.6 万 kWh 降至 10.3 万 kWh，紧急购电费从 111.32 万元降至 61.76 万元，降幅 44.5%。这表明增大安全余量能有效降低高价紧急购电风险。

（4）**总费呈 U 型分布**

总费用先降后升，在 $Q_{SAFE} = 0.95$ 处达到最低值 1447.76 万元。这体现了安全余量的边际收益递减：

$Q_{SAFE} < 0.95$ 时，增大安全余量带来的紧急费节省大于计划费增加；

$Q_{SAFE} > 0.95$ 时，继续增大安全余量，计划费增加超过紧急费节省。

综上，$Q_{SAFE} = 0.95$ 是本模型的最优安全分位数。此时总费用 1447.76 万元，比 $Q_{SAFE} = 0.70$ 低 11.21 万元（降幅 0.77%），比 $Q_{SAFE} = 0.99$ 低 4.55 万元（降幅 0.31%）。因此本文选择 $Q_{SAFE} = 0.95$ 作为最终参数。

5.3.5 引入其他时刻预报的必要性分析

根据问题三的求解结果，本文统计了全年调整购电量绝对值和紧急购电量在各

时段的分布。结果表明，调整购电量主要集中在 14:00–18:00 时段。该区间位于 12:00 和 18:00 两次预报之间，主要受 12:00 预报调整策略的影响。12:00 预报在 14:00 时已使用 2 小时，在 16:00 时已使用 4 小时，在 18:00 时已使用 6 小时，信息时效性逐渐下降。

进一步观察该时段的紧急购电情况：由结果表明 14:00–18:00 同时出现较大的调整量和较多的紧急购电，则说明仅依靠 12:00 预报难以及时修正光伏偏差，建议在 12:00 和 18:00 之间增加一次 15:00 预报，将最长信息间隔从 6 小时缩短为 3 小时，从而降低该时段的调整费用和紧急购电费用。因此，建议在 15:00 增加一次预报。

5.4 问题四模型的建立与求解

5.4.1 模型的建立

相对于前三问，本问更加接近现实情境，外部电价会随时间波动，附件 4 提供了 2025 年全年每天的实时电价，每个时段一个价格，因此在建立模型之前需要对电价进行假设，在每天 0:00 时，当天 144 个时段的电价已由外网公布，决策者可获取当天实时电价序列 c_t，但无法预知未来日期的电价。因此每天的购电策略仅基于当天电价制定，逐日滚动优化。

本问与前三问存在一定内部联系，综合小区负载、光伏发电效率、外部波动电价等多个条件及前三问的求解方法，得出本问目标函数：

问题 4-2（对应问题 2）的计划阶段目标函数为：

$$\min \sum_{t=1}^{T} c_t \, P_t^{buy} \Delta t \tag{5.32}$$

问题 4-3（对应问题 3）在调整阶段的目标函数为：

$$\min \sum_{t=\tau}^{T} [1.5 c_t dP_t{}^{+} + 0.5 c_t dP_t{}^{-}] \; \Delta t \tag{5.33}$$

紧急购电费用：

$$C_{emg} = \sum_{t=1}^{T} 5 \, c_t P_t^{emg} \Delta t \tag{5.34}$$

调整费用：

$$C_{\text{adj}} = \sum_{t=1}^{T} [1.5 c_t \max(P_t^{eff} - P_t^{plan}, 0) + 0.5 c_t \max(P_t^{plan} - P_t^{eff}, 0)] \; \Delta t \tag{5.35}$$

总费用：

$$C_{total} = \sum_{t} c_t P_t^{plan} \Delta t + C_{adj} + C_{emg} \tag{5.36}$$

同时，也可得出结论，本问约束条件与问题二、三完全一致，包括功率平衡约束、储能电量动态约束、储能上下限约束、充放电功率约束、购电量非负约束。此处不再重复。

相较于前三问的固定电价模型，本问中采用的实时电价模型具有以下新特征：

（1）购电时段分布成为决策变量：优化器会主动将购电量向低价时段倾斜。

（2）储能套利成为核心机制：低价时段多充、高价时段多放，赚取峰谷价差。

（3）价格风险显著增加：高价时段的紧急购电成本为 5c_t，约为低价时段的 10﹣15 倍，优化器必须在 “套利收益” 与 “紧急购电风险” 之间权衡。

5.4.2 问题四结果分析

表 5.7 对比了在电价变化的情况下，其他决策量的增幅情况

表 5.7 四种方案全年购电费用与单位购电成本对比

指标	问题 2	问题 3	问题 4-2	问题 4-3
计划购电费用/万元	1 463.52	1 294.79	1 523.10	1 350.01
调整购电费用/万元	—	69.94	—	74.34
紧急购电费用/万元	29.84	83.03	36.21	87.87
全年总费用/万元	1 493.35	1 447.76	1 559.31	1 512.22
单位购电成本/(元/kWh)	0.8324	0.8070	0.8692	0.8429

固定电价场景中，问题 2 计划购电费为 1463.52 万元，引入滚动调整后的问题 3 计划购电费降至 1294.79 万元，成本降低 168.73 万元；实时电价场景中，问题 4-2 计划购电费为 1523.10 万元，问题 4-3 计划购电费降至 1350.01 万元，成本降低 173.09 万元。

可以看出，无论固定电价还是实时电价，滚动调整策略均会大幅压低日前计划购电费用。核心机理为：多阶段滚动调整为系统预留了日内修正空间，因此日前计划无需过度保守，可采用更激进的购电策略、适度压低日前计划购电量，从源头削减基础购电成本，这也是滚动优化实现降本的核心原因

成本结构数据显示，固定电价下问题 3 全年调整费用为 69.94 万元，实时电价下问题 4-3 调整费用为 74.34 万元，实时电价场景调整费用小幅增加 4.4 万元。

该现象符合运行逻辑：实时电价存在动态波动，叠加光伏出力实时偏差，系统

日内功率供需调节空间更大、调整频次与调整体量更高，因此产生了少量额外调整成本，但该部分成本增量远小于计划购电费的节省额度，不影响整体降本效果。

固定电价场景下，问题 2 紧急购电费为 29.84 万元，问题 3 紧急购电费升至 83.03 万元，增幅 53.19 万元；实时电价场景下，问题 4-2 紧急购电费为 36.21 万元，问题 4-3 紧急购电费升至 87.87 万元，增幅 51.66 万元。

滚动调整策略会导致紧急购电成本小幅上升，其本质是策略的经济性取舍：日前计划主动减少购电量，会小幅提升日内功率缺口与紧急购电概率，造成紧急费用增加。但从整体效益来看，紧急购电费约 50 万元的增量，远低于计划购电费约 170 万元的降幅，最终实现全年总费用的显著下降。

综上，本文所提多阶段滚动调整策略具备极强的经济理性，通过可控、小幅增加极端场景下的紧急购电风险，换取了大规模的日前购电成本压降，在风险可控的前提下实现了微网全年购电效益最优。

六、 模型的评价

6.1 模型的优点

1. 本模型所采用策略具有低价充能、放电套利、光伏消纳等特征。在夜间谷电时间段（0:00-4:00），储能设备以最大功率（5000 kW）充电，利用低价电力填充储能。此阶段不放电，储能电量从 6000 kWh 上升至约 10050 kWh。随后，电价在 4：00-8：00 开始上升，此阶段储能大量放电（约释放 6365.84kWh）以满足小区负载需求，同时为了避免高价购电，在此阶段进行外网购电，补充储能设备电能

2. 本模型在设计与求解层面展现出良好的综合性能。首先，模型严格遵循因果性原则，预测与误差估计均仅依赖当日之前的历史数据，契合“提前一天制定购电计划”的实际调度场景，确保了策略的可实施性。同时，通过引入日末储备约束，模型以少量计划成本换取了稳定的应急缓冲能力，显著削减了夜间紧急购电的发生频率与成本。在求解性能方面，该模型基于线性规划（LP）框架，保证了每日调度计划的全局最优性；在稀疏约束条件下，全年多场景组合求解可在分钟级完成，具备优异的计算效率与可扩展性。此外，风险参数 q 可灵活调节，便于开展灵敏度分析，为管理决策提供了多维度的参考依据。

6.2 模型的缺点

负载预测仅使用历史负载，光伏预测仅使用历史光伏，温度、云量、辐照度、节假日等强相关变量均未纳入模型。改进方向是引入气象预报数据作为回归特征，或将预测模块升级为 LSTM、XGBoost 等机器学习时序模型，以提高预测精度。

七、 AI 工具使用声明

本参赛队在竞赛过程中使用了 AI 工具，主要用于代码调试、图表绘制与论文排版，详细使用情况见支撑材料。

参考文献

[1]粘建军,李世田.清洁能源发电企业高质量发展的"四个转变"[J].企业管理, 2020(S02):168-169.

[2]Mitchell S, O'Sullivan M, Dunning I. PuLP: a linear programming toolkit for Python[EB/OL]. (2011)[2026-09-13]. https://coin-or.github.io/pulp/.

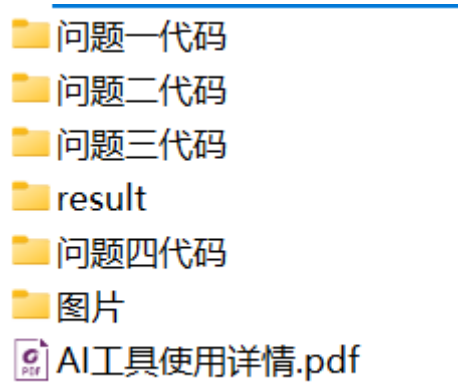
[3] Forrest J, Lougee-Heimer R. CBC user guide[M]//Emerging theory, methods, and applications. Hanover: INFORMS, 2005: 257-277.

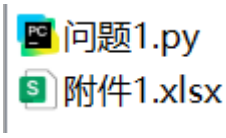
[4] Bertsimas D, Sim M. The price of robustness[J]. Operations Research, 2004, 52(1): 35-53.

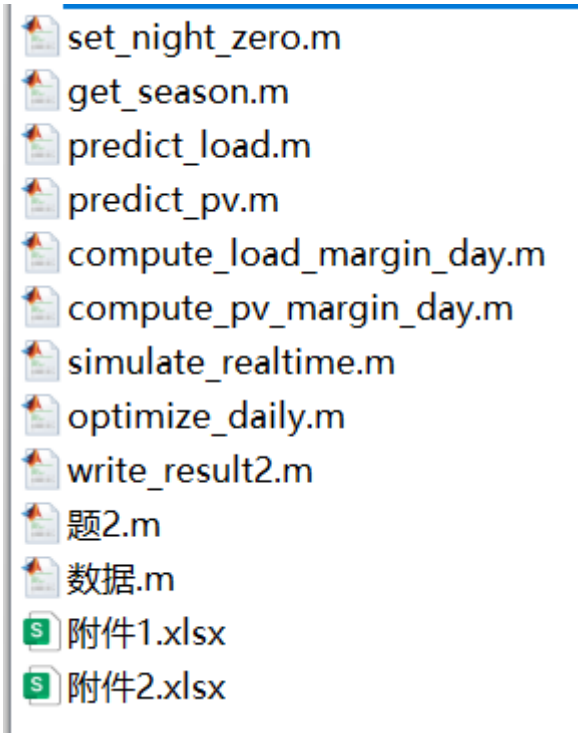
[5] 智谱 AI. GLM 大语言模型（智谱清言）[CP/OL]. (2025)[2026-09-13]. https://chatglm.cn.

附录

支撑材料的文件列表



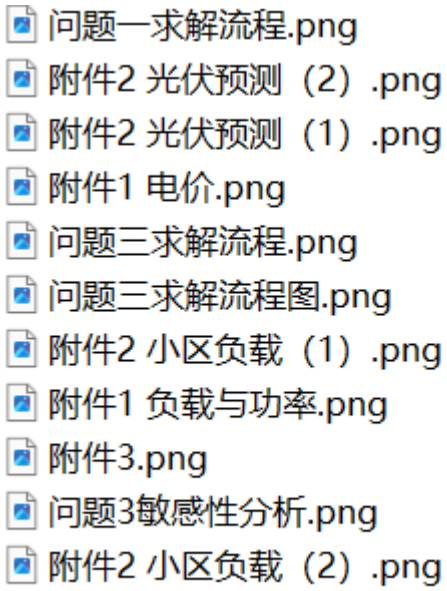




get_season.m
predict_load_q3.m
staircase_pv_fc_stage.m
staircase_pv_fc.m
compute_pv_margin_q3.m
compute_pv_bias_q3.m
compute_load_margin_day.m
simulate_realtime3.m
solve_stage0.m
solve_stageK.m
write_result3.m
题4_3.m
附件1.xlsx
附件3.xlsx
附件4.xlsx
附件2.xlsx

result1.xlsx
result4-2.xlsx
result2.xlsx
result3.xlsx
result4-3.xlsx

问题一求解流程.png
附件2 光伏预测（2）.png
附件2 光伏预测（1）.png
附件1 电价.png
问题三求解流程.png
问题三求解流程图.png
附件2 小区负载（1）.png
附件1 负载与功率.png
附件3.png
问题3敏感性分析.png
附件2 小区负载（2）.png



附录 1　问题一的求解代码（Python）

```
import openpyxl
import pulp
from openpyxl.styles import Font, Alignment, Border, Side, PatternFill
def load_data(filepath):
    wb = openpyxl.load_workbook(filepath, data_only=True)
    ws = wb['Sheet1']
    rows = list(ws.iter_rows(values_only=True))
    data = rows[1:]
    T = len(data)
    dt = 10 / 60
    times_str = []
    prices, loads, pvs = [], [], []
    for r in data:
        t = r[0]
        if isinstance(t, str):
            if '+1' in t or '0:00+1' in t:
                times_str.append("24:00")
            else:
                times_str.append(t)
        else:
            times_str.append(f"{t.hour:02d}:{t.minute:02d}")
        prices.append(float(r[1]))
        loads.append(float(r[2]))
        pvs.append(float(r[3]))
    return times_str, prices, loads, pvs, T, dt
E_MAX = 10800.0
E_MIN = 1200.0
E_INIT = 6000.0
```

```
    P_CH_MAX = 5000.0
    P_DIS_MAX = 5000.0
    ETA_CH = 0.90
    ETA_DIS = 0.90
    def solve_microgrid(prices, loads, pvs, T, dt):
        prob = pulp.LpProblem("Microgrid_Problem1", pulp.LpMinimize)
        P_buy = [pulp.LpVariable(f'P_buy_{t}', lowBound=0) for t in range(T)]
        P_ch = [pulp.LpVariable(f'P_ch_{t}', lowBound=0, upBound=P_CH_MAX) for t in
range(T)]
        P_dis = [pulp.LpVariable(f'P_dis_{t}', lowBound=0, upBound=P_DIS_MAX) for t in
range(T)]
        E = [pulp.LpVariable(f'E_{t}', lowBound=E_MIN, upBound=E_MAX) for t in range(T +
1)]
        prob += pulp.lpSum(prices[t] * P_buy[t] * dt for t in range(T)), "Total_Cost"
        for t in range(T):
            prob += P_buy[t] + pvs[t] + P_dis[t] == loads[t] + P_ch[t],
f'Power_Balance_{t}'
        prob += E[0] == E_INIT, 'E_initial'
        for t in range(T):
            prob += E[t+1] == E[t] + ETA_CH * P_ch[t] * dt - P_dis[t] * dt / ETA_DIS,
f'SOC_Dynamics_{t}'
        prob += E[T] == E_INIT, 'E_final'
        prob.solve(pulp.PULP_CBC_CMD(msg=0))
        if prob.status != pulp.LpStatusOptimal:
            raise RuntimeError("模型求解失败！请检查约束条件。")
        P_buy_val = [P_buy[t].varValue for t in range(T)]
        P_ch_val = [P_ch[t].varValue for t in range(T)]
        P_dis_val = [P_dis[t].varValue for t in range(T)]
        E_val = [E[t].varValue for t in range(T + 1)]
        return prob, P_buy_val, P_ch_val, P_dis_val, E_val
    def verify_results(P_buy, P_ch, P_dis, E, prices, loads, pvs, T, dt):
        balance_ok = True
        for t in range(T):
            lhs = P_buy[t] + pvs[t] + P_dis[t]
            rhs = loads[t] + P_ch[t]
            if abs(lhs - rhs) > 0.01:
                balance_ok = False
        capacity_ok = all(E_MIN - 0.01 <= e <= E_MAX + 0.01 for e in E)
        power_ok = all(0 - 0.01 <= p <= P_CH_MAX + 0.01 for p in P_ch) and \
                   all(0 - 0.01 <= p <= P_DIS_MAX + 0.01 for p in P_dis)
        cycle_ok = abs(E[0] - E[T]) < 0.01
        nonneg_ok = all(p >= -0.01 for p in P_buy)
        total_buy = sum(P_buy) * dt
        total_pv = sum(pvs) * dt
        total_load = sum(loads) * dt
        total_ch = sum(P_ch) * dt
```

```
        total_dis = sum(P_dis) * dt
        ch_loss = total_ch * (1 - ETA_CH)
        dis_loss = total_dis * (1/ETA_DIS - 1)
        energy_in = total_buy + total_pv
        energy_out = total_load + ch_loss + dis_loss
        energy_balance_ok = abs(energy_in - energy_out) < 1.0
        all_ok = balance_ok and capacity_ok and power_ok and cycle_ok and nonneg_ok and
energy_balance_ok
        return all_ok
    def generate_tables(times_str, prices, loads, pvs, P_buy, P_ch, P_dis, E, T, dt):
        table1_periods = [
            ("10:00-10:10", "10:10"),
            ("12:00-12:10", "12:10"),
            ("14:00-14:10", "14:10"),
            ("16:00-16:10", "16:10"),
            ("18:00-18:10", "18:10"),
            ("20:00-20:10", "20:10"),
        ]
        table1_data = []
        for period_name, time_tag in table1_periods:
            idx = times_str.index(time_tag)
            buy_kwh = P_buy[idx] * dt
            table1_data.append((period_name, buy_kwh))
        total_buy = sum(P_buy) * dt
        total_cost = sum(prices[t] * P_buy[t] * dt for t in range(T))
        table2_periods = [
            ("0:00-4:00",   0,  24),
            ("4:00-8:00",   24, 48),
            ("8:00-12:00",  48, 72),
            ("12:00-16:00", 72, 96),
            ("16:00-20:00", 96, 120),
            ("20:00-24:00", 120, 144),
        ]
        table2_data = []
        for period_name, start, end in table2_periods:
            ch_kwh = sum(P_ch[t] * dt for t in range(start, end))
            dis_kwh = sum(P_dis[t] * dt for t in range(start, end))
            table2_data.append((period_name, ch_kwh, dis_kwh))
        return {
            'table1': table1_data,
            'table2': table2_data,
            'total_buy': total_buy,
            'total_cost': total_cost,
            'E_0': E[0],
            'E_24': E[T],
```

```
        }
    def save_to_excel(times_str, prices, loads, pvs, P_buy, P_ch, P_dis, E, T, dt,
tables):
        header_font = Font(name='宋体', size=11, bold=True, color='FFFFFF')
        header_fill = PatternFill(start_color='4472C4', end_color='4472C4',
fill_type='solid')
        center = Alignment(horizontal='center', vertical='center')
        thin_border = Border(
            left=Side(style='thin'),
            right=Side(style='thin'),
            top=Side(style='thin'),
            bottom=Side(style='thin')
        )
        wb = openpyxl.Workbook()
        ws1 = wb.active
        ws1.title = "表 1_购电量"
        ws1['A1'] = "表 1 微网在指定时间段的购电量及全天的购电量和购电费"
        ws1.merge_cells('A1:B1')
        ws1['A1'].font = Font(name='宋体', size=12, bold=True)
        ws1['A1'].alignment = center
        headers1 = ['时间段', '购电量(kWh)']
        for col, h in enumerate(headers1, 1):
            c = ws1.cell(row=2, column=col, value=h)
            c.font = header_font
            c.alignment = center
            c.border = thin_border
            c.fill = header_fill
        for i, (period, buy_kwh) in enumerate(tables['table1']):
            ws1.cell(row=3+i, column=1, value=period).alignment = center
            ws1.cell(row=3+i, column=2, value=round(buy_kwh, 4)).alignment = center
            ws1.cell(row=3+i, column=1).border = thin_border
            ws1.cell(row=3+i, column=2).border = thin_border
        row_total = 3 + len(tables['table1'])
        ws1.cell(row=row_total, column=1, value="全天购电量(kWh)").alignment = center
        ws1.cell(row=row_total, column=2, value=round(tables['total_buy'], 4)).alignment
= center
        ws1.cell(row=row_total+1, column=1, value="全天购电费(元)").alignment = center
        ws1.cell(row=row_total+1, column=2, value=round(tables['total_cost'],
4)).alignment = center
        for r in [row_total, row_total+1]:
            for c in [1, 2]:
                ws1.cell(row=r, column=c).border = thin_border
                ws1.cell(row=r, column=c).font = Font(name='宋体', size=11, bold=True)
        ws1.column_dimensions['A'].width = 22
        ws1.column_dimensions['B'].width = 20
        ws2 = wb.create_sheet("表 2_储能充放电")
```

```
        ws2['A1'] = "表 2 储能设备在指定时间段的充放电量及 0:00 和 24:00 的储电量"
        ws2.merge_cells('A1:C1')
        ws2['A1'].font = Font(name='宋体', size=12, bold=True)
        ws2['A1'].alignment = center
        headers2 = ['时间段', '充电量(kWh)', '放电量(kWh)']
        for col, h in enumerate(headers2, 1):
            c = ws2.cell(row=2, column=col, value=h)
            c.font = header_font
            c.alignment = center
            c.border = thin_border
            c.fill = header_fill
        for i, (period, ch_kwh, dis_kwh) in enumerate(tables['table2']):
            ws2.cell(row=3+i, column=1, value=period).alignment = center
            ws2.cell(row=3+i, column=2, value=round(ch_kwh, 4)).alignment = center
            ws2.cell(row=3+i, column=3, value=round(dis_kwh, 4)).alignment = center
            for c in [1, 2, 3]:
                ws2.cell(row=3+i, column=c).border = thin_border
        row_end = 3 + len(tables['table2'])
        ws2.cell(row=row_end, column=1, value="0:00 储电量(kWh)").alignment = center
        ws2.cell(row=row_end, column=2, value=round(tables['E_0'], 4)).alignment = center
        ws2.cell(row=row_end+1, column=1, value="24:00 储电量(kWh)").alignment = center
        ws2.cell(row=row_end+1, column=2, value=round(tables['E_24'], 4)).alignment =
center
        for r in [row_end, row_end+1]:
            for c in [1, 2, 3]:
                ws2.cell(row=r, column=c).border = thin_border
                ws2.cell(row=r, column=c).font = Font(name='宋体', size=11, bold=True)
        ws2.column_dimensions['A'].width = 22
        ws2.column_dimensions['B'].width = 20
        ws2.column_dimensions['C'].width = 20
        ws3 = wb.create_sheet("完整策略")
        headers3 = ['时间段', '电价(元/kWh)', '负载(kW)', '光伏(kW)',
                    '购电量(kWh)', '充电量(kWh)', '放电量(kWh)', '储电量(kWh)', '购电费
(元)']
        for col, h in enumerate(headers3, 1):
            c = ws3.cell(row=1, column=col, value=h)
            c.font = header_font
            c.alignment = center
            c.border = thin_border
            c.fill = header_fill
        for t in range(T):
            row = 2 + t
            ws3.cell(row=row, column=1, value=times_str[t])
            ws3.cell(row=row, column=2, value=round(prices[t], 4))
            ws3.cell(row=row, column=3, value=round(loads[t], 4))
```

```
            ws3.cell(row=row, column=4, value=round(pvs[t], 4))
            ws3.cell(row=row, column=5, value=round(P_buy[t] * dt, 4))
            ws3.cell(row=row, column=6, value=round(P_ch[t] * dt, 4))
            ws3.cell(row=row, column=7, value=round(P_dis[t] * dt, 4))
            ws3.cell(row=row, column=8, value=round(E[t], 4))
            ws3.cell(row=row, column=9, value=round(prices[t] * P_buy[t] * dt, 4))
            for c in range(1, 10):
                ws3.cell(row=row, column=c).alignment = center
                ws3.cell(row=row, column=c).border = thin_border
        for col_letter in ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I']:
            ws3.column_dimensions[col_letter].width = 16
        output_path = 'result1.xlsx'
        wb.save(output_path)
    def calc_no_ess_cost(prices, loads, pvs, T, dt):
        cost_no_ess = 0
        for t in range(T):
            net = loads[t] - pvs[t]
            if net > 0:
                cost_no_ess += prices[t] * net * dt
        return cost_no_ess
    def main():
        data_file = '附件1.xlsx'
        times_str, prices, loads, pvs, T, dt = load_data(data_file)
        prob, P_buy, P_ch, P_dis, E = solve_microgrid(prices, loads, pvs, T, dt)
        verify_results(P_buy, P_ch, P_dis, E, prices, loads, pvs, T, dt)
        tables = generate_tables(times_str, prices, loads, pvs, P_buy, P_ch, P_dis, E, T,
dt)
        save_to_excel(times_str, prices, loads, pvs, P_buy, P_ch, P_dis, E, T, dt,
tables)
        cost_no_ess = calc_no_ess_cost(prices, loads, pvs, T, dt)
        cost_with_ess = tables['total_cost']
        saving = cost_no_ess - cost_with_ess
        saving_pct = saving / cost_no_ess * 100
        print(f"未调整策略前的总价：{cost_no_ess:.2f} 元")
        print(f"调整后的总价：      {cost_with_ess:.2f} 元")
        print(f"下降的价格：       {saving:.2f} 元")
        print(f"下降的百分比：      {saving_pct:.2f}%")
    if __name__ == "__main__":
        main()
```

附录 2　问题二的求解代码（MATLAB）

```
clear; clc; close all;
```

```
N = 144; dt = 1/6; eta = 0.9;
soc0 = 6000; soc_Min = 1200; soc_Max = 10800; p_Max = 5000;
qSafe = 0.93;

predStart = datetime(2025,2,1);
predEnd = datetime(2025,12,31);

price_Table = readtable('附件 1.xlsx', 'VariableNamingRule', 'preserve');
price = price_Table{:, 2}; price = price(:);

loadTable = readtable('附件 2.xlsx', 'Sheet', '小区负载', ...
    'VariableNamingRule', 'preserve');
loadDates = dateshift(datetime(loadTable{:,1}), 'start', 'day');
loadData = loadTable{:, 2:145};

pvTable = readtable('附件 2.xlsx', 'Sheet', '光伏发电实际功率', ...
    'VariableNamingRule', 'preserve');
pvDates = dateshift(datetime(pvTable{:,1}), 'start', 'day');
pvData = pvTable{:, 2:145};

fprintf('数据加载完成\n');
fprintf('电价: %d 个点\n', length(price));
fprintf('负载: %d 天 × %d 时段\n', size(loadData,1), size(loadData,2));
fprintf('光伏: %d 天 × %d 时段\n', size(pvData,1), size(pvData,2));

predDates = (predStart:caldays(1):predEnd)';
nDays = length(predDates);
fprintf('预测天数: %d\n', nDays);

combos = {
    '组合 1: 负载 A(上周同期) + 光伏 A(前日同期)', 'A', 'A';
    '组合 2: 负载 A(上周同期) + 光伏 B(近 7 天加权)', 'A', 'B';
    '组合 3: 负载 B(同季度同星期) + 光伏 A(前日同期)', 'B', 'A';
    '组合 4: 负载 B(同季度同星期) + 光伏 B(近 7 天加权)', 'B', 'B';
    };

comboResults = cell(4,1);
comboCosts = zeros(4,3);

for c = 1:4
    comboName = combos{c,1};
    loadMethod = combos{c,2};
```

```
pvMethod = combos{c,3};
fprintf('\n=== %s ===\n', comboName);

results = struct([]);
costPlan = 0; costEmg = 0;
socNow = soc0;

for i = 1:nDays
    d = predDates(i);

    loadPred = predict_load(d, loadDates, loadData, loadMethod);
    pvPred = predict_pv(d, pvDates, pvData, pvMethod);
    if isempty(loadPred) || isempty(pvPred)
        continue;
    end

    deltaPv = compute_pv_margin_day(d, pvDates, pvData, pvMethod, qSafe);
    deltaLd = compute_load_margin_day(d, loadDates, loadData, loadMethod, qSafe);

    loadPlan = loadPred + deltaLd;
    pvPlan = max(pvPred - deltaPv, 0);

    if i < nDays
        socEndMin = soc0;
    else
        socEndMin = 0;
    end

    [pBuy, pCh, pDis, soc] = optimize_daily(...
        loadPlan, pvPlan, price, N, dt, eta, socNow, ...
        soc_Min, soc_Max, p_Max, socEndMin);
    if isempty(pBuy)
        fprintf(' 优化失败: %s\n', datestr(d));
        continue;
    end

    idx = find(loadDates == d, 1);
    if isempty(idx)
        continue;
    end
    loadAct = loadData(idx,:)';
    pvAct = pvData(idx,:)';
```

```
        [emg, dayCostPlan, dayCostEmg, socHist] = simulate_realtime(...
            pBuy, pCh, pDis, loadAct, pvAct, price, dt, ...
            socNow, eta, soc_Min, soc_Max, p_Max);

        soc(end) = socHist(end);

        results(end+1).date = d;
        results(end).pBuy = pBuy;
        results(end).pCh = pCh;
        results(end).pDis = pDis;
        results(end).soc = soc;
        results(end).emg = emg;
        results(end).costPlan = dayCostPlan;
        results(end).costEmg = dayCostEmg;
        results(end).socInit = socNow;

        costPlan = costPlan + dayCostPlan;
        costEmg = costEmg + dayCostEmg;
        socNow = socHist(end);

        if mod(i,50)==0 || i==1
            fprintf(' [%d/%d] %s 计划费=%.0f 紧急费=%.0f SOC=%.1f\n', ...
                i, nDays, datestr(d), dayCostPlan, dayCostEmg, socNow);
        end
    end

    comboResults{c} = results;
    comboCosts(c,:) = [costPlan, costEmg, costPlan + costEmg];
    fprintf('总计: 计划费=%.0f 紧急费=%.0f 总费=%.0f\n', ...
        costPlan, costEmg, costPlan + costEmg);
end

[~, bestIdx] = min(comboCosts(:,3));
fprintf('\n=== 所有组合对比 ===\n');
for c = 1:4
    fprintf('%-50s 计划费=%10.0f 紧急费=%10.0f 总费=%10.0f\n', ...
        combos{c,1}, comboCosts(c,1), comboCosts(c,2), comboCosts(c,3));
end
fprintf('\n★ 最优组合: %s\n', combos{bestIdx,1});
fprintf(' 总成本: %.0f 元\n', comboCosts(bestIdx,3));

write_result2(comboResults{bestIdx}, N, dt);
fprintf('\n 完成!\n');
```

```
function Delta = compute_load_margin_day(d, date, data, method, q)
Ncol = size(data,2);
idx = find(date < d);
if numel(idx) < 8
    Delta = zeros(Ncol,1);
    return;
end

predd = NaN(numel(idx), Ncol);
act  = data(idx, :);

all_season = arrayfun(@(x) get_season(x.Month), date);
wd_all = weekday(date);

for j = 1:numel(idx)
    k = idx(j);
    if strcmp(method, 'A')
        tgt = k - 7;
        if tgt >= 1 && days(date(k) - date(tgt)) == 7
            predd(j,:) = data(tgt,:);
        end
    else
        hit = idx(idx < k);
        m = hit(all_season(hit)==all_season(k) & wd_all(hit)==wd_all(k));
        if isempty(m)
            m = hit(wd_all(hit)==wd_all(k));
        end
        if ~isempty(m)
            predd(j,:) = mean(data(m,:),1);
        end
    end
end

valid = ~isnan(predd(:,1));
if nnz(valid) < 5
    Delta = zeros(Ncol,1);
    return;
end
err = max(predd(valid,:) - act(valid,:), 0);
Delta = quantile(err, q)';
end

function margin = compute_pv_margin_day(d, date, data, method, q)
```

```
nCol = size(data,2);
idx = find(date < d);
if numel(idx) < 8
    margin = zeros(nCol,1);
    return;
end

pred = NaN(numel(idx), nCol);
act  = data(idx, :);

if strcmp(method, 'A')
    for j = 1:numel(idx)
        k = idx(j);
        tgt = k - 1;
        if tgt >= 1 && days(date(k) - date(tgt)) == 1
            pred(j,:) = set_night_zero(data(tgt,:)');
        end
    end
else
    w = [0.30 0.25 0.20 0.15 0.07 0.02 0.01];
    for j = 1:numel(idx)
        k = idx(j);
        acc = zeros(1, nCol); wSum = 0;
        for m = 1:7
            tgt = k - m;
            if tgt >= 1
                acc = acc + w(m)*data(tgt,:);
                wSum = wSum + w(m);
            end
        end
        if wSum > 0
            pred(j,:) = set_night_zero(acc / wSum);
        end
    end
end

valid = ~isnan(pred(:,1));
if nnz(valid) < 5
    margin = zeros(nCol,1);
    return;
end
err = max(pred(valid,:) - act(valid,:), 0);
margin = quantile(err, q)';
```

```
end

function s = get_season(monthNum)
    if ismember(monthNum, [3 4 5])
        s = 1;
    elseif ismember(monthNum, [6 7 8])
        s = 2;
    elseif ismember(monthNum, [9 10 11])
        s = 3;
    else
        s = 4;
    end
end

function [pBuy, pCh, pDis, soc] = optimize_daily(...
    loadPred, pvPred, price, N, dt, eta, socInit, socMin, socMax, pMax, socEndMin)

if nargin < 11
    socEndMin = 0;
end

nVar = 5 * N;

f = zeros(nVar,1);
f(1:N) = price * dt;

nEq = 2 * N;
Aeq = sparse(nEq, nVar);
beq = zeros(nEq, 1);

for t = 1:N
    Aeq(t, t) = 1;
    Aeq(t, N+t) = -1;
    Aeq(t, 2*N+t) = 1;
    Aeq(t, 4*N+t) = -1;
    beq(t) = loadPred(t) - pvPred(t);
end

for t = 1:N
    row = N + t;
    Aeq(row, 3*N+t) = 1;
    Aeq(row, N+t) = -eta*dt;
    Aeq(row, 2*N+t) = dt/eta;
```

```
    if t > 1
        Aeq(row, 3*N+t-1) = -1;
    else
        beq(row) = socInit;
    end
end

if socEndMin > socMin
    nUb = 2*N + 1;
else
    nUb = 2*N;
end
Aub = sparse(nUb, nVar);
bUb = zeros(nUb, 1);
for t = 1:N
    Aub(t, 3*N+t) = 1;      bUb(t) = socMax;
    Aub(N+t, 3*N+t) = -1;   bUb(N+t) = -socMin;
end
if socEndMin > socMin
    Aub(nUb, 3*N+N) = -1;
    bUb(nUb) = -socEndMin;
end

lb = zeros(nVar,1);
ub = inf(nVar,1);
ub(N+1:2*N) = pMax;
ub(2*N+1:3*N) = pMax;
lb(3*N+1:4*N) = socMin;
ub(3*N+1:4*N) = socMax;

options = optimoptions('linprog', 'Display', 'off');
[x, ~, exitflag] = linprog(f, Aub, bUb, Aeq, beq, lb, ub, options);

if exitflag == 1
    pBuy = max(x(1:N), 0);
    pCh = max(x(N+1:2*N), 0);
    pDis = max(x(2*N+1:3*N), 0);
    soc = x(3*N+1:4*N);
else
    pBuy = []; pCh = []; pDis = []; soc = [];
end
end

function pred = predict_load(d, dates, data, method)
```

```
    pred = [];
    if strcmp(method, 'A')
        target = d - caldays(7);
        idx = find(dates == target, 1);
        if ~isempty(idx)
            pred = data(idx,:)';
        end
    elseif strcmp(method, 'B')
        season = get_season(d.Month);
        wd = weekday(d);

        seasons = arrayfun(@(x) get_season(x.Month), dates);
        weekdays = weekday(dates);

        mask = (dates < d) & (seasons == season) & (weekdays == wd);
        idx = find(mask);

        if isempty(idx)
            mask = (dates < d) & (weekdays == wd);
            idx = find(mask);
        end

        if ~isempty(idx)
            pred = mean(data(idx,:), 1)';
        end
    end
end

function pred = predict_pv(d, dates, data, method)
    pred = [];
    if strcmp(method, 'A')
        target = d - caldays(1);
        idx = find(dates == target, 1);
        if ~isempty(idx)
            pred = data(idx,:)';
            pred = set_night_zero(pred);
        end
    elseif strcmp(method, 'B')
        w = [0.30 0.25 0.20 0.15 0.07 0.02 0.01];
        pred = zeros(144,1);
        wSum = 0;
        for k = 1:7
            target = d - caldays(k);
            idx = find(dates == target, 1);
```

```
            if ~isempty(idx)
                pred = pred + w(k) * data(idx,:)';
                wSum = wSum + w(k);
            end
        end
        if wSum > 0
            pred = pred / wSum;
            pred = set_night_zero(pred);
        else
            pred = [];
        end
    end
end

function pred = set_night_zero(pred)
    pred(1:29) = 0;
    pred(113:144) = 0;
end

function [emg, costPlan, costEmg, socHist] = simulate_realtime(...
        pBuy, pCh, pDis, loadAct, pvAct, price, dt, ...
        socInit, eta, socMin, socMax, pMax)

    N = length(pBuy);
    emg = zeros(N,1);
    socHist = zeros(N,1);
    soc = socInit;

    for t = 1:N
        available = pvAct(t) + pBuy(t);
        need = loadAct(t);

        if available >= need
            surplus = available - need;

            chLimP = pMax;
            chLimE = (socMax - soc) / (eta * dt);
            chNow = min([surplus, chLimP, chLimE]);
            chNow = max(chNow, 0);

            disNow = 0;
            emg(t) = 0;
        else
```

```
            deficit = need - available;

            chNow = 0;

            disLimP = pMax;
            disLimE = (soc - socMin) * eta / dt;
            disNow = min([deficit, disLimP, disLimE]);
            disNow = max(disNow, 0);

            deficit = deficit - disNow;

            emg(t) = max(deficit, 0);
        end

        soc = soc + eta * chNow * dt - disNow * dt / eta;
        soc = max(min(soc, socMax), socMin);
        socHist(t) = soc;
    end

    costPlan = sum(price .* pBuy * dt);
    costEmg  = sum(5 * price .* emg * dt);
end

function write_result2(results, N, dt)
    nDays = length(results);
    if nDays == 0
        warning('没有结果可写入');
        return;
    end

    timeLabels = cell(1, N);
    for t = 1:N
        startMin = (t-1)*10;
        endMin   = t*10;
        sh = floor(startMin/60); sm = mod(startMin,60);
        eh = floor(endMin/60);   em = mod(endMin,60);
        if t == N
            timeLabels{t} = sprintf('%d:%02d-%d:00+1', sh, sm, eh);
        else
            timeLabels{t} = sprintf('%d:%02d-%d:%02d', sh, sm, eh, em);
        end
    end
```

```
sheet1Rows = cell(nDays, 147);
for i = 1:nDays
    r = results(i);
    sheet1Rows{i,1} = datestr(r.date, 'yyyy-mm-dd HH:MM:SS');
    buyRotated = [r.pBuy(2:end); r.pBuy(1)];
    for t = 1:N
        sheet1Rows{i, t+1} = round(buyRotated(t), 4);
    end
    sheet1Rows{i,146} = round(sum(r.pBuy) * dt, 4);
    sheet1Rows{i,147} = round(r.costPlan, 4);
end
writecell(sheet1Rows, 'result2.xlsx', 'Sheet', '计划购电量', 'Range', 'A2');

periodRanges = [1 24; 25 48; 49 72; 73 96; 97 120; 121 144];
periodLabels = {'0:00-4:00', '4:00-8:00', '8:00-12:00', ...
                '12:00-16:00', '16:00-20:00', '20:00-24:00'};
nRows2 = nDays * 6;
sheet2_data = cell(nRows2, 6);
row = 1;
for i = 1:nDays
    r = results(i);
    dateStr = datestr(r.date, 'yyyy-mm-dd');
    for p = 1:6
        rng = periodRanges(p,1):periodRanges(p,2);
        chVal  = sum(r.pCh(rng))  * dt;
        disVal = sum(r.pDis(rng)) * dt;

        if p == 1
            sheet2_data{row,1} = dateStr;
        else
            sheet2_data{row,1} = '';
        end
        sheet2_data{row,2} = periodLabels{p};
        sheet2_data{row,3} = round(chVal, 4);
        sheet2_data{row,4} = round(disVal, 4);

        if p == 1
            sheet2_data{row,5} = '00:00:00';
            sheet2_data{row,6} = round(r.socInit, 4);
        elseif p == 2
            sheet2_data{row,5} = '24:00';
            sheet2_data{row,6} = round(r.soc(end), 4);
        else
            sheet2_data{row,5} = '';
```

```
                sheet2_data{row,6} = '';
            end
            row = row + 1;
        end
    end
    writecell(sheet2_data, 'result2.xlsx', 'Sheet', '充放电量', 'Range', 'A2');

    nEmgRows = 0;
    for i = 1:nDays
        nEmgRows = nEmgRows + sum(results(i).emg > 0.01);
    end

    if nEmgRows == 0
        sheet3Rows = {'无', '无', 0};
    else
        sheet3Rows = cell(nEmgRows, 3);
        row = 1;
        for i = 1:nDays
            r = results(i);
            dateStr = datestr(r.date, 'yyyy-mm-dd');
            firstOfDay = true;
            for t = 1:N
                if r.emg(t) > 0.01
                    if firstOfDay
                        sheet3Rows{row,1} = dateStr;
                        firstOfDay = false;
                    else
                        sheet3Rows{row,1} = '';
                    end
                    sheet3Rows{row,2} = timeLabels{t};
                    sheet3Rows{row,3} = round(r.emg(t) * dt, 4);
                    row = row + 1;
                end
            end
        end
    end
    writecell(sheet3Rows, 'result2.xlsx', 'Sheet', '紧急购电量', 'Range', 'A2');

    fprintf('result2.xlsx 已保存（%d 天）\n', nDays);
end
```

附录 3　问题三的求解代码（MATLAB）

```
clear; clc; close all;
if exist('result3.xlsx', 'file'), delete('result3.xlsx'); end
```

```
N = 144; dt = 1/6; eta = 0.9;
soc0 = 6000; socMin = 1200; socMax = 10800; pMax = 5000;
qSafe = 0.95;
usePvMargin = true;
predStart = datetime(2025,2,1);
predEnd   = datetime(2025,12,31);



priceTable = readtable('附件 1.xlsx', 'VariableNamingRule', 'preserve');
priceRaw = priceTable{:, 2}; priceRaw = priceRaw(:);
if length(priceRaw) == 24
    price = repelem(priceRaw, 6);
elseif length(priceRaw) == 144
    price = priceRaw;
else
    error('附件 1 电价长度异常：%d', length(priceRaw));
end


loadTable = readtable('附件 2.xlsx', 'Sheet', '小区负载', 'VariableNamingRule',
'preserve');
loadDates = dateshift(datetime(loadTable{:,1}), 'start', 'day');
loadData  = loadTable{:, 2:145};


pvTable = readtable('附件 2.xlsx', 'Sheet', '光伏发电实际功率', 'VariableNamingRule',
'preserve');
pvDates = dateshift(datetime(pvTable{:,1}), 'start', 'day');
pvData  = pvTable{:, 2:145};


fcRaw = readmatrix('附件 3.xlsx');
if isempty(fcRaw), error('读取附件 3 失败'); end
nFcRows = size(fcRaw, 1);
nFcDays = floor(nFcRows / 4);
fcData = cell(nFcDays, 1);
for d0 = 1:nFcDays
    rows = (d0-1)*4 + (1:4);
    fcData{d0} = fcRaw(rows, end-23:end);
end
fcBaseDate = datetime(2025,1,1);
fcDates = (predStart:caldays(1):predEnd)';
nDays = length(fcDates);
fprintf('数据加载完成: 优化 %d 天, 预报覆盖 %d 天\n', nDays, nFcDays);


dayResults = cell(nDays, 1);
```

```
totalCost = 0;
socNow = soc0;
nAdjusted = 0;

diagPlanTotal = 0; diagNeedTotal = 0; diagEmgTotal = 0;
diagEmgCost = 0; diagPlanCost = 0; diagAdjCost = 0;
diagNoAdjCost = 0; diagPvBiasSum = 0; diagLdBiasSum = 0;
diagCnt = 0;

for i = 1:nDays
    d = fcDates(i);
    iFc = days(d - fcBaseDate) + 1;
    if iFc < 1 || iFc > nFcDays, continue; end
    fc = fcData{iFc};
    if iFc > 1
        prevFc = fcData{iFc-1};
    else
        prevFc = NaN(4, 24);
    end

    loadPredRaw = predict_load_q3(d, loadDates, loadData);
    if isempty(loadPredRaw), continue; end

    idxAct = find(loadDates == d, 1);
    if ~isempty(idxAct)
        loadAct = loadData(idxAct, :)';
        pvAct   = pvData(idxAct, :)';
    else
        loadAct = loadPredRaw;
        pvAct   = staircase_pv_fc(fc(1,:), prevFc(1,:));
    end
    isLastDay = (d == predEnd);

    deltaLd = compute_load_margin_day(d, loadDates, loadData, 'A', qSafe);
    if usePvMargin
        deltaPv = compute_pv_bias_q3(d, fcBaseDate, fcData, pvDates, pvData, 30);
    else
        deltaPv = zeros(N, 1);
    end

    pvPlanRaw = staircase_pv_fc(fc(1,:), prevFc(1,:));
    pvPlan = max(pvPlanRaw - deltaPv, 0);
```

```
nightMask = (pvPlanRaw < 50);
deltaLdUse = deltaLd;
deltaLdUse(~nightMask) = 0;
loadPred = loadPredRaw + deltaLdUse;

[pPlan, ~, ~, ~] = solve_stage0(...
    loadPred, pvPlan, price, N, dt, eta, socNow, ...
    socMin, socMax, pMax, isLastDay);
if isempty(pPlan)
    fprintf(' [优化失败] %s 阶段 0\n', datestr(d));
    continue;
end
pEff = pPlan;

[~,~,~,soc36] = simulate_realtime3(pPlan(1:36), ...
    loadAct(1:36), pvAct(1:36), price(1:36), 36, dt, eta, ...
    socNow, socMin, socMax, pMax);
socAt36 = soc36(end);

pv6Raw = staircase_pv_fc_stage(fc(2,:), 37);
pv6 = max(pv6Raw - deltaPv(37:N), 0);
nightMask6 = (pv6Raw < 50);
deltaLd6 = deltaLd(37:N);
deltaLd6(~nightMask6) = 0;
loadPred6 = loadPredRaw(37:N) + deltaLd6;

[pAdj6, ~, ~, ~, ~, ~] = solve_stageK(...
    loadPred6, pv6, price(37:N), N-36, dt, eta, socAt36, ...
    socMin, socMax, pMax, pPlan(37:N), isLastDay);
if ~isempty(pAdj6), pEff(37:N) = pAdj6; end

pUpto72 = [pPlan(1:36); pEff(37:72)];
[~,~,~,soc72] = simulate_realtime3(pUpto72, ...
    loadAct(1:72), pvAct(1:72), price(1:72), 72, dt, eta, ...
    socNow, socMin, socMax, pMax);
socAt72 = soc72(end);

pv12Raw = staircase_pv_fc_stage(fc(3,:), 73);
pv12 = max(pv12Raw - deltaPv(73:N), 0);
nightMask12 = (pv12Raw < 50);
deltaLd12 = deltaLd(73:N);
deltaLd12(~nightMask12) = 0;
loadPred12 = loadPredRaw(73:N) + deltaLd12;
```

```
[pAdj12, ~, ~, ~, ~, ~] = solve_stageK(...
    loadPred12, pv12, price(73:N), N-72, dt, eta, socAt72, ...
    socMin, socMax, pMax, pPlan(73:N), isLastDay);
if ~isempty(pAdj12), pEff(73:N) = pAdj12; end

pUpto108 = [pPlan(1:36); pEff(37:72); pEff(73:108)];
[~,~,~,soc108] = simulate_realtime3(pUpto108, ...
    loadAct(1:108), pvAct(1:108), price(1:108), 108, dt, eta, ...
    socNow, socMin, socMax, pMax);
socAt108 = soc108(end);

pv18Raw = staircase_pv_fc_stage(fc(4,:), 109);
pv18 = max(pv18Raw - deltaPv(109:N), 0);
nightMask18 = (pv18Raw < 50);
deltaLd18 = deltaLd(109:N);
deltaLd18(~nightMask18) = 0;
loadPred18 = loadPredRaw(109:N) + deltaLd18;

[pAdj18, ~, ~, ~, ~, ~] = solve_stageK(...
    loadPred18, pv18, price(109:N), N-108, dt, eta, socAt108, ...
    socMin, socMax, pMax, pPlan(109:N), isLastDay);
if ~isempty(pAdj18), pEff(109:N) = pAdj18; end

[emg, pChSim, pDisSim, socTrace, costEmg] = simulate_realtime3(...
    pEff, loadAct, pvAct, price, N, dt, eta, socNow, ...
    socMin, socMax, pMax);

[emgNoAdj, pChNoAdj, pDisNoAdj, socNoAdj, costEmgNoAdj] = ...
    simulate_realtime3(pPlan, loadAct, pvAct, price, N, dt, eta, ...
    socNow, socMin, socMax, pMax);

costPlan = sum(price .* pPlan) * dt;
dP = pEff - pPlan;
costAdjWith = sum(1.5*price.*max(dP,0) + 0.5*price.*max(-dP,0)) * dt;
costTotalWith = costPlan + costAdjWith + costEmg;
costTotalNo   = costPlan + costEmgNoAdj;

if costTotalWith <= costTotalNo
    costAdjFinal  = costAdjWith;
    costEmgFinal  = costEmg;
    pChUse = pChSim; pDisUse = pDisSim;
    socUse  = socTrace; emgUse = emg;
```

```
    costTotal = costTotalWith;
    usedAdj = true;
else
    pEff = pPlan;
    costAdjFinal = 0;
    costEmgFinal = costEmgNoAdj;
    pChUse = pChNoAdj; pDisUse = pDisNoAdj;
    socUse  = socNoAdj;  emgUse  = emgNoAdj;
    costTotal = costTotalNo;
    usedAdj = false;
end
totalCost = totalCost + costTotal;
netAdj = pEff - pPlan;
if usedAdj, nAdjusted = nAdjusted + 1; end

dayResults{i}.date = d;
dayResults{i}.pPlan = pPlan;
dayResults{i}.pEff = pEff;
dayResults{i}.netAdj = netAdj;
dayResults{i}.pCh = pChUse;
dayResults{i}.pDis = pDisUse;
dayResults{i}.emg = emgUse;
dayResults{i}.socTrace = socUse;
dayResults{i}.costPlan = costPlan;
dayResults{i}.costAdj = costAdjFinal;
dayResults{i}.costEmg = costEmgFinal;
dayResults{i}.costTotal = costTotal;
dayResults{i}.socInit = socNow;
socNow = socUse(end);

planKwh = sum(pPlan) * dt;
needKwh = sum(loadAct - pvAct) * dt;
emgKwh  = sum(emgUse) * dt;
pvBiasDay = mean(pvPlan(43:102) - pvAct(43:102));
ldBiasDay = mean(loadPred - loadAct);
diagPlanTotal = diagPlanTotal + planKwh;
diagNeedTotal = diagNeedTotal + needKwh;
diagEmgTotal  = diagEmgTotal + emgKwh;
diagEmgCost   = diagEmgCost + costEmgFinal;
diagPlanCost  = diagPlanCost + costPlan;
diagAdjCost   = diagAdjCost + costAdjFinal;
diagNoAdjCost = diagNoAdjCost + costEmgNoAdj;
diagPvBiasSum = diagPvBiasSum + pvBiasDay;
diagLdBiasSum = diagLdBiasSum + ldBiasDay;
```

```
    diagCnt = diagCnt + 1;

    if mod(i,20)==0 || i==1 || i==nDays
        if usedAdj, flagStr = '调整'; else, flagStr = '不调整'; end
        fprintf(' [%d/%d] %s 计划=%.0f 调整=%.0f 紧急=%.0f 总=%.0f SOC=%.1f %s\n', ...
            i, nDays, datestr(d), costPlan, costAdjFinal, ...
            costEmgFinal, costTotal, socNow, flagStr);
    end
end

fprintf('\n========================================\n');
fprintf('总费用: %.0f 元\n', totalCost);
fprintf('========================================\n');



fprintf('统计天数: %d\n', diagCnt);
fprintf('方案 A 被选中: %d 天 (%.1f%%)\n', nAdjusted, 100*nAdjusted/max(diagCnt,1));
fprintf('计划购电量总计: %.0f kWh\n', diagPlanTotal);
fprintf('实际需求总计: %.0f kWh\n', diagNeedTotal);
fprintf('差额（计划-需求）: %.0f kWh\n', diagPlanTotal - diagNeedTotal);
fprintf('紧急购电量总计: %.0f kWh (%.2f%%)\n', ...
    diagEmgTotal, 100*diagEmgTotal/max(diagNeedTotal,1));
fprintf('计划费: %.0f 元\n', diagPlanCost);
fprintf('调整费: %.0f 元\n', diagAdjCost);
fprintf('实际总费: %.0f 元\n', totalCost);
fprintf('光伏预报偏差: %.2f kW\n', diagPvBiasSum / max(diagCnt,1));
fprintf('负载预报偏差: %.2f kW\n', diagLdBiasSum / max(diagCnt,1));



write_result3(dayResults, N, dt);
fprintf('result3.xlsx 已保存\n');

function margin = compute_load_margin_day(d, dates, data, method, q)

nCol = size(data,2);
idx = find(dates < d);
if numel(idx) < 8
    margin = zeros(nCol,1);
    return;
end

pred = NaN(numel(idx), nCol);
```

```
act  = data(idx, :);

seasons = arrayfun(@(x) get_season(x.Month), dates);
weekdays = weekday(dates);

for j = 1:numel(idx)
    k = idx(j);
    if strcmp(method, 'A')
        tgt = k - 7;
        if tgt >= 1 && days(dates(k) - dates(tgt)) == 7
            pred(j,:) = data(tgt,:);
        end
    else
        hist = idx(idx < k);
        m = hist(seasons(hist)==seasons(k) & weekdays(hist)==weekdays(k));
        if isempty(m)
            m = hist(weekdays(hist)==weekdays(k));
        end
        if ~isempty(m)
            pred(j,:) = mean(data(m,:),1);
        end
    end
end

valid = ~isnan(pred(:,1));
if nnz(valid) < 5
    margin = zeros(nCol,1);
    return;
end
err = max(pred(valid,:) - act(valid,:), 0);
margin = quantile(err, q)';
end

function bias = compute_pv_bias_q3(d, fcBaseDate, fcData, pvDates, pvData, window)

N = 144;
bias = zeros(N, 1);

idx = find(pvDates < d);
if numel(idx) < 8
    return;
end
```

```
idx = idx(max(1, end-window+1):end);

pred = NaN(numel(idx), N);
act  = NaN(numel(idx), N);

for j = 1:numel(idx)
    k = idx(j);
    dk = pvDates(k);

    iFc = days(dk - fcBaseDate) + 1;
    if iFc < 1 || iFc > numel(fcData)
        continue;
    end

    if iFc > 1
        prevFc = fcData{iFc-1};
    else
        prevFc = NaN(4,24);
    end

    fc0 = fcData{iFc};
    pred(j, :) = staircase_pv_fc(fc0(1,:), prevFc(1,:))';
    act(j, :)  = pvData(k, :);
end

valid = ~isnan(pred(:,1)) & ~isnan(act(:,1));
if nnz(valid) < 5
    return;
end

bias = mean(pred(valid,:) - act(valid,:), 1)';
end

function margin = compute_pv_margin_q3(d, fcBaseDate, fcData, pvDates, pvData, q)

N = 144;
margin = zeros(N, 1);

idx = find(pvDates < d);
if numel(idx) < 8
    return;
end
```

```
pred = NaN(numel(idx), N);
act  = NaN(numel(idx), N);

for j = 1:numel(idx)
    k = idx(j);
    dk = pvDates(k);

    iFc = days(dk - fcBaseDate) + 1;
    if iFc < 1 || iFc > numel(fcData)
        continue;
    end

    if iFc > 1
        prevFc = fcData{iFc-1};
    else
        prevFc = NaN(4,24);
    end

    fc0 = fcData{iFc};
    pred(j, :) = staircase_pv_fc(fc0(1,:), prevFc(1,:))';
    act(j, :)  = pvData(k, :);
end

valid = ~isnan(pred(:,1)) & ~isnan(act(:,1));
if nnz(valid) < 5
    return;
end

err = max(pred(valid,:) - act(valid,:), 0);
margin = quantile(err, q)';
end

function s = get_season(month_num)
    if ismember(month_num, [3 4 5])
        s = 1;
    elseif ismember(month_num, [6 7 8])
        s = 2;
    elseif ismember(month_num, [9 10 11])
        s = 3;
    else
        s = 4;
    end
end
```

```
function pred = predict_load_q3(targetDate, histDates, histData)
    N = size(histData, 2);
    refDate = targetDate - days(7);
    idx = find(histDates == refDate, 1);
    if isempty(idx)
        pred = [];
        return;
    end
    pred = histData(idx, :)';
end



function [emg, pCh, pDis, socTrace, costEmg] = simulate_realtime3(...
    pBuy, loadAct, pvAct, price, N, dt, eta, socInit, ...
    socMin, socMax, pMax)

emg       = zeros(N, 1);
pCh      = zeros(N, 1);
pDis     = zeros(N, 1);
socTrace = zeros(N, 1);
soc = socInit;

for t = 1:N
    supply = pvAct(t) + pBuy(t);
    demand = loadAct(t);

    if supply >= demand
        surplus    = supply - demand;
        chLimE = (socMax - soc) / (eta * dt);
        chNow  = min([surplus, pMax, chLimE]);
        chNow  = max(chNow, 0);

        pCh(t) = chNow;
        soc = soc + chNow * eta * dt;
    else
        deficit     = demand - supply;
        disLimE = (soc - socMin) * eta / dt;
        disNow  = min([deficit, pMax, disLimE]);
        disNow  = max(disNow, 0);

        pDis(t) = disNow;
```

```
        soc = soc - disNow * dt / eta;

        remaining = deficit - disNow;
        emg(t) = max(remaining, 0);
    end

    soc = max(min(soc, socMax), socMin);
    socTrace(t) = soc;
end

costEmg = sum(5 * price .* emg) * dt;
end

function [pPlan, pCh, pDis, soc] = solve_stage0(...
    loadPred, pvPred, price, N, dt, eta, socInit, ...
    socMin, socMax, pMax, isLastDay)

nVar = 5 * N;

f = zeros(nVar, 1);
f(1:N) = price * dt;

nEq = 3 * N;
Aeq = zeros(nEq, nVar);
beq = zeros(nEq, 1);

for t = 1:N
    row = t;
    Aeq(row, t) = 1;
    Aeq(row, N+t) = -1;
    Aeq(row, 2*N+t) = 1;
    Aeq(row, 3*N+t) = -1;
    beq(row) = loadPred(t) - pvPred(t);
end

for t = 1:N
    row = N + t;
    Aeq(row, 2*N+t) = dt / eta;
    Aeq(row, N+t) = -eta * dt;
    Aeq(row, 4*N+t) = 1;
    if t == 1
        beq(row) = socInit;
    else
```

```
        Aeq(row, 4*N+t-1) = -1;
    end
end

socReserve = 4500;
Aineq = zeros(1, nVar);
Aineq(5*N) = -1;
bineq = -socReserve;

lb = zeros(nVar, 1);
ub = inf(nVar, 1);
lb(4*N+1 : 5*N) = socMin;
ub(4*N+1 : 5*N) = socMax;
ub(N+1 : 2*N) = pMax;
ub(2*N+1 : 3*N) = pMax;

options = optimoptions('linprog', 'Display', 'off', 'Algorithm', 'dual-simplex');
[x, ~, exitflag] = linprog(f, Aineq, bineq, Aeq, beq, lb, ub, options);

if exitflag <= 0
    warning('阶段 0 优化未收敛，exitflag = %d', exitflag);
    pPlan = []; pCh = []; pDis = []; soc = [];
    return;
end

pPlan = x(1:N);
pCh = x(N+1 : 2*N);
pDis = x(2*N+1 : 3*N);
soc = x(4*N+1 : 5*N);
end

function [pBuy, pCh, pDis, soc, dPup, dPdown] = solve_stageK(...
    loadPred, pvPred, price, M, dt, eta, socInit, ...
    socMin, socMax, pMax, pPlanSeg, isLastDay)

nVar = 7 * M;

maxReachSoc = socInit;
for t = 1:M
    maxReachSoc = min(maxReachSoc + pMax * eta * dt, socMax);
end
if isLastDay
    socReserveTarget = min(6000, maxReachSoc);
```

```
else
    socReserveTarget = [];
end

f = zeros(nVar, 1);
f(5*M+1:6*M) = 1.5 * price(1:M) * dt;
f(6*M+1:7*M) = 0.5 * price(1:M) * dt;

nEq = 3 * M;
Aeq = sparse(nEq, nVar);
beq = zeros(nEq, 1);

for t = 1:M
    row = t;
    Aeq(row, t) = 1;
    Aeq(row, M+t) = -1;
    Aeq(row, 2*M+t) = 1;
    Aeq(row, 3*M+t) = -1;
    beq(row) = loadPred(t) - pvPred(t);
end

for t = 1:M
    row = M + t;
    Aeq(row, 4*M+t) = 1;
    Aeq(row, M+t) = -eta * dt;
    Aeq(row, 2*M+t) = dt / eta;
    if t > 1
        Aeq(row, 4*M+t-1) = -1;
    else
        beq(row) = socInit;
    end
end

for t = 1:M
    row = 2*M + t;
    Aeq(row, t) = 1;
    Aeq(row, 5*M+t) = -1;
    Aeq(row, 6*M+t) = 1;
    beq(row) = pPlanSeg(t);
end

nUb = 2 * M;
if ~isempty(socReserveTarget), nUb = nUb + 1; end
Aub = sparse(nUb, nVar);
```

```
bUb = zeros(nUb, 1);
rowNum = 0;
for t = 1:M
    rowNum = rowNum + 1;
    Aub(rowNum, 4*M+t) = 1;  bUb(rowNum) = socMax;
    rowNum = rowNum + 1;
    Aub(rowNum, 4*M+t) = -1; bUb(rowNum) = -socMin;
end
if ~isempty(socReserveTarget)
    rowNum = rowNum + 1;
    Aub(rowNum, 4*M+M) = -1;
    bUb(rowNum) = -socReserveTarget;
end

lb = zeros(nVar, 1);
ub = inf(nVar, 1);
ub(M+1:2*M) = pMax;
ub(2*M+1:3*M) = pMax;
lb(4*M+1:5*M) = socMin;
ub(4*M+1:5*M) = socMax;

options = optimoptions('linprog', 'Display', 'off', 'Algorithm', 'dual-simplex');
[x, ~, exitflag] = linprog(f, Aub, bUb, Aeq, beq, lb, ub, options);

if exitflag == 1
    pBuy  = max(x(1:M), 0);
    pCh   = max(x(M+1:2*M), 0);
    pDis  = max(x(2*M+1:3*M), 0);
    soc      = x(4*M+1:5*M);
    dPup = max(x(5*M+1:6*M), 0);
    dPdown = max(x(6*M+1:7*M), 0);
else
    pBuy = []; pCh = []; pDis = []; soc = []; dPup = []; dPdown = [];
end
end

function pv = staircase_pv_fc(fc24, fc24Prev)

N = 144;
pv = zeros(N, 1);

if ~isempty(fc24Prev) && ~isnan(fc24Prev(24))
    pv(1:6) = max(fc24Prev(24), 0);
end
```

```
for h = 1:24
    startIdx = (h-1)*6 + 7;
    endIdx   = min(startIdx + 5, N);
    if startIdx <= N
        pv(startIdx:endIdx) = max(fc24(h), 0);
    end
end
end

function pv = staircase_pv_fc_stage(fc24, tStart)

N = 144;
pv = zeros(N - tStart + 1, 1);

for h = 1:24
    absStart = tStart + (h-1)*6;
    for s = 0:5
        target = absStart + s;
        if target >= tStart && target <= N
            pv(target - tStart + 1) = max(fc24(h), 0);
        end
    end
end
end

function write_result3(all_results, N, dt)
valid   = ~cellfun(@isempty, all_results);
results = all_results(valid);
n_days  = length(results);

if n_days == 0
    warning('没有结果可写入');
    return;
end

% ========== 生成 144 个时间标签 ==========
time_labels = cell(1, N);
for t = 1:N
    start_min = (t-1)*10;
    end_min   = t*10;
    sh = floor(start_min/60); sm = mod(start_min, 60);
    eh = floor(end_min/60);   em = mod(end_min, 60);
```

```
    if t == N
        time_labels{t} = sprintf('%d:%02d-%d:00+1', sh, sm, eh);
    else
        time_labels{t} = sprintf('%d:%02d-%d:%02d', sh, sm, eh, em);
    end
end

if exist('result3.xlsx', 'file')
    delete('result3.xlsx');
end

%% ---------- Sheet1: 计划购电量 ----------
header1 = cell(1, N + 2);
header1{1} = '日期\时间';
for t = 1:N, header1{t+1} = time_labels{t}; end
header1{N+1} = '全天购电量';
header1{N+2} = '全天购电费';
writecell(header1, 'result3.xlsx', 'Sheet', '计划购电量', 'Range', 'A1');

sheet1_data = cell(n_days, N + 2);
for i = 1:n_days
    r = results{i};
    sheet1_data{i,1} = datestr(r.date, 'yyyy-mm-dd');
    for t = 1:N
        sheet1_data{i, t+1} = round(r.P_plan(t) * dt, 4);   % ★ 转 kWh
    end
    sheet1_data{i,N+1} = round(sum(r.P_plan) * dt, 4);
    sheet1_data{i,N+2} = round(r.cost_plan, 4);
end
writecell(sheet1_data, 'result3.xlsx', 'Sheet', '计划购电量', 'Range', 'A2');

%% ---------- Sheet2: 调整购电量 ----------
header2 = cell(1, N + 2);
header2{1} = '日期\时间';
for t = 1:N, header2{t+1} = time_labels{t}; end
header2{N+1} = '全天购电量';
header2{N+2} = '全天购电费';
writecell(header2, 'result3.xlsx', 'Sheet', '调整购电量', 'Range', 'A1');

sheet2_data = cell(n_days, N + 2);
for i = 1:n_days
    r = results{i};
    sheet2_data{i,1} = datestr(r.date, 'yyyy-mm-dd');
```

```
    for t = 1:N
        sheet2_data{i, t+1} = round(r.net_adj(t) * dt, 4);  % ★ 转 kWh
    end
    sheet2_data{i,N+1} = round(sum(r.net_adj) * dt, 4);
    sheet2_data{i,N+2} = round(r.cost_adj, 4);
end
writecell(sheet2_data, 'result3.xlsx', 'Sheet', '调整购电量', 'Range', 'A2');

%% ---------- Sheet3: 充放电量 ----------
% ★ 题目表 2：6 个 4 小时时段
header3 = {'日期', '时间段', '充电量', '放电量', '时刻', '储电量'};
writecell(header3, 'result3.xlsx', 'Sheet', '充放电量', 'Range', 'A1');

period_ranges = [1 24; 25 48; 49 72; 73 96; 97 120; 121 144];
period_labels = {'0:00-4:00','4:00-8:00','8:00-12:00', ...
                 '12:00-16:00','16:00-20:00','20:00-24:00'};
n_rows3 = n_days * 6;
sheet3_data = cell(n_rows3, 6);
row = 1;
for i = 1:n_days
    r = results{i};
    date_str = datestr(r.date, 'yyyy-mm-dd');
    for p = 1:6
        rng = period_ranges(p,1):period_ranges(p,2);
        ch_val  = sum(r.P_ch(rng))  * dt;
        dis_val = sum(r.P_dis(rng)) * dt;

        if p == 1
            sheet3_data{row,1} = date_str;
        else
            sheet3_data{row,1} = '';
        end
        sheet3_data{row,2} = period_labels{p};
        sheet3_data{row,3} = round(ch_val, 4);
        sheet3_data{row,4} = round(dis_val, 4);

        if p == 1
            sheet3_data{row,5} = '00:00:00';
            sheet3_data{row,6} = round(r.SOC_init, 4);
        elseif p == 6
            sheet3_data{row,5} = '24:00';
            sheet3_data{row,6} = round(r.SOC_trace(end), 4);
        else
            sheet3_data{row,5} = '';
```

```
            sheet3_data{row,6} = '';
        end
        row = row + 1;
    end
end
writecell(sheet3_data, 'result3.xlsx', 'Sheet', '充放电量', 'Range', 'A2');

%% ---------- Sheet4: 紧急购电量 ----------
header4 = {'日期', '购电时间段', '购电量'};
writecell(header4, 'result3.xlsx', 'Sheet', '紧急购电量', 'Range', 'A1');

total_emg_rows = 0;
for i = 1:n_days
    total_emg_rows = total_emg_rows + sum(results{i}.emg > 0.01);
end
if total_emg_rows == 0
    sheet4_data = {'无', '无', 0};
else
    sheet4_data = cell(total_emg_rows, 3);
    row = 1;
    for i = 1:n_days
        r = results{i};
        date_str = datestr(r.date, 'yyyy-mm-dd');
        first_in_day = true;
        for t = 1:N
            if r.emg(t) > 0.01
                if first_in_day
                    sheet4_data{row,1} = date_str;
                    first_in_day = false;
                else
                    sheet4_data{row,1} = '';
                end
                sheet4_data{row,2} = time_labels{t};
                sheet4_data{row,3} = round(r.emg(t) * dt, 4);
                row = row + 1;
            end
        end
    end
end
writecell(sheet4_data, 'result3.xlsx', 'Sheet', '紧急购电量', 'Range', 'A2');

fprintf('result3.xlsx 已保存（%d 天）\n', n_days);
end
```

附录 4　问题 4-2 的求解代码（MATLAB）

　　问题 4-3 的求解代码（MATLAB）

```
clear; clc; close all;

N = 144; dt = 1/6; eta = 0.9;
soc0 = 6000; socMin = 1200; socMax = 10800; pMax = 5000;
qSafe = 0.93;

PRED_START = datetime(2025,2,1);
PRED_END = datetime(2025,12,31);

price4Table = readtable('附件 4.xlsx', 'VariableNamingRule', 'preserve');
price4Dates = dateshift(datetime(price4Table{:,1}), 'start', 'day');
price4Data  = price4Table{:, 2:145};

loadTable = readtable('附件 2.xlsx', 'Sheet', '小区负载', ...
    'VariableNamingRule', 'preserve');
loadDates = dateshift(datetime(loadTable{:,1}), 'start', 'day');
loadData = loadTable{:, 2:145};

pvTable = readtable('附件 2.xlsx', 'Sheet', '光伏发电实际功率', ...
    'VariableNamingRule', 'preserve');
pvDates = dateshift(datetime(pvTable{:,1}), 'start', 'day');
pvData = pvTable{:, 2:145};

fprintf('数据加载完成\n');
fprintf('实时电价: %d 天 × %d 时段\n', size(price4Data,1), size(price4Data,2));
fprintf('负载: %d 天 × %d 时段\n', size(loadData,1), size(loadData,2));
fprintf('光伏: %d 天 × %d 时段\n', size(pvData,1), size(pvData,2));

predDates = (PRED_START:caldays(1):PRED_END)';
nDays = length(predDates);
fprintf('预测天数: %d\n', nDays);

combos = {
    '组合 1: 负载 A(上周同期) + 光伏 A(前日同期)', 'A', 'A';
    '组合 2: 负载 A(上周同期) + 光伏 B(近 7 天加权)', 'A', 'B';
    '组合 3: 负载 B(同季度同星期) + 光伏 A(前日同期)', 'B', 'A';
    '组合 4: 负载 B(同季度同星期) + 光伏 B(近 7 天加权)', 'B', 'B';
    };

comboResults = cell(4,1);
```

```
comboCosts = zeros(4,3);

for c = 1:4
    comboName = combos{c,1};
    loadMethod = combos{c,2};
    pvMethod = combos{c,3};
    fprintf('\n=== %s ===\n', comboName);

    results = struct([]);
    costPlan = 0; costEmg = 0;
    socNow = soc0;

    for i = 1:nDays
        d = predDates(i);

        idxP = find(price4Dates == d, 1);
        if isempty(idxP)
            fprintf(' [跳过] %s 无附件 4 电价\n', datestr(d));
            continue;
        end
        price = price4Data(idxP, :)';

        loadPred = predict_load(d, loadDates, loadData, loadMethod);
        pvPred = predict_pv(d, pvDates, pvData, pvMethod);
        if isempty(loadPred) || isempty(pvPred)
            continue;
        end

        deltaPv = compute_pv_margin_day(d, pvDates, pvData, pvMethod, qSafe);
        deltaLd = compute_load_margin_day(d, loadDates, loadData, loadMethod, qSafe);

        loadPlan = loadPred + deltaLd;
        pvPlan = max(pvPred - deltaPv, 0);

        if i < nDays
            socEndMin = 1200;
        else
            socEndMin = soc0;
        end

        [pBuy, pCh, pDis, soc] = optimize_daily(...
            loadPlan, pvPlan, price, N, dt, eta, socNow, ...
            socMin, socMax, pMax, socEndMin);
```

```
        if isempty(pBuy)
            fprintf(' 优化失败: %s\n', datestr(d));
            continue;
        end

        idx = find(loadDates == d, 1);
        if isempty(idx)
            continue;
        end
        loadAct = loadData(idx,:)';
        pvAct = pvData(idx,:)';

        [emg, dayCostPlan, dayCostEmg, socHist] = simulate_realtime(...
            pBuy, pCh, pDis, loadAct, pvAct, price, dt, ...
            socNow, eta, socMin, socMax, pMax);

        soc(end) = socHist(end);

        results(end+1).date = d;
        results(end).pBuy = pBuy;
        results(end).pCh = pCh;
        results(end).pDis = pDis;
        results(end).soc = soc;
        results(end).emg = emg;
        results(end).dayCostPlan = dayCostPlan;
        results(end).dayCostEmg = dayCostEmg;
        results(end).socInit = socNow;

        costPlan = costPlan + dayCostPlan;
        costEmg = costEmg + dayCostEmg;
        socNow = socHist(end);

        if mod(i,50)==0 || i==1
            fprintf(' [%d/%d] %s 计划费=%.0f 紧急费=%.0f SOC=%.1f\n', ...
                i, nDays, datestr(d), dayCostPlan, dayCostEmg, socNow);
        end
    end

    comboResults{c} = results;
    comboCosts(c,:) = [costPlan, costEmg, costPlan + costEmg];
    fprintf('总计: 计划费=%.0f 紧急费=%.0f 总费=%.0f\n', ...
        costPlan, costEmg, costPlan + costEmg);
end
```

```
[~, bestIdx] = min(comboCosts(:,3));
fprintf('\n=== 所有组合对比 ===\n');
for c = 1:4
    fprintf('%-50s 计划费=%10.0f 紧急费=%10.0f 总费=%10.0f\n', ...
        combos{c,1}, comboCosts(c,1), comboCosts(c,2), comboCosts(c,3));
end
fprintf('\n★ 最优组合: %s\n', combos{bestIdx,1});
fprintf(' 总成本: %.0f 元\n', comboCosts(bestIdx,3));

write_result2(comboResults{bestIdx}, N, dt, 'result4-2.xlsx');
fprintf('\n 完成!\n');

function margin = compute_load_margin_day(d, date, data, method, q)
nCol = size(data,2);
idx = find(date < d);
if numel(idx) < 8
    margin = zeros(nCol,1);
    return;
end

pred = NaN(numel(idx), nCol);
act  = data(idx, :);

seasons = arrayfun(@(x) get_season(x.Month), date);
weekdays = weekday(date);

for j = 1:numel(idx)
    k = idx(j);
    if strcmp(method, 'A')
        tgt = k - 7;
        if tgt >= 1 && days(date(k) - date(tgt)) == 7
            pred(j,:) = data(tgt,:);
        end
    else
        hit = idx(idx < k);
        m = hit(seasons(hit)==seasons(k) & weekdays(hit)==weekdays(k));
        if isempty(m)
            m = hit(weekdays(hit)==weekdays(k));
        end
        if ~isempty(m)
            pred(j,:) = mean(data(m,:),1);
        end
```

```
    end
end

valid = ~isnan(pred(:,1));
if nnz(valid) < 5
    margin = zeros(nCol,1);
    return;
end
err = max(pred(valid,:) - act(valid,:), 0);
margin = quantile(err, q)';
end



function margin = compute_pv_margin_day(d, date, data, method, q)

ncol = size(data,2);
idx = find(date < d);
if numel(idx) < 8
    margin = zeros(ncol,1);
    return;
end

pred = NaN(numel(idx), ncol);
act  = data(idx, :);

if strcmp(method, 'A')
    for j = 1:numel(idx)
        k = idx(j);
        tgt = k - 1;
        if tgt >= 1 && days(date(k) - date(tgt)) == 1
            pred(j,:) = set_night_zero(data(tgt,:)');
        end
    end
else
    w = [0.30 0.25 0.20 0.15 0.07 0.02 0.01];
    for j = 1:numel(idx)
        k = idx(j);
        a_cc = zeros(1, ncol); wSum = 0;
        for m = 1:7
            tgt = k - m;
            if tgt >= 1
                a_cc = a_cc + w(m)*data(tgt,:);
                wSum = wSum + w(m);
            end
```

```
        end
        if wSum > 0
            pred(j,:) = set_night_zero(a_cc / wSum);
        end
    end
end

valid = ~isnan(pred(:,1));
if nnz(valid) < 5
    margin = zeros(ncol,1);
    return;
end
err = max(pred(valid,:) - act(valid,:), 0);
margin = quantile(err, q)';
end

function s = get_season(monthNum)
    if ismember(monthNum, [3 4 5])
        s = 1;
    elseif ismember(monthNum, [6 7 8])
        s = 2;
    elseif ismember(monthNum, [9 10 11])
        s = 3;
    else
        s = 4;
    end
end

function [pBuy, pCh, pDis, soc] = optimize_daily(...
    loadPred, pvPred, price, N, dt, eta, socInit, socMin, socMax, pMax, socEndMin)

if nargin < 11
    socEndMin = 0;
end

nVar = 5 * N;

f = zeros(nVar,1);
f(1:N) = price * dt;

nEq = 2 * N;
Aeq = sparse(nEq, nVar);
beq = zeros(nEq, 1);
```

```
for t = 1:N
    Aeq(t, t) = 1;
    Aeq(t, N+t) = -1;
    Aeq(t, 2*N+t) = 1;
    Aeq(t, 4*N+t) = -1;
    beq(t) = loadPred(t) - pvPred(t);
end

for t = 1:N
    row = N + t;
    Aeq(row, 3*N+t) = 1;
    Aeq(row, N+t) = -eta*dt;
    Aeq(row, 2*N+t) = dt/eta;
    if t > 1
        Aeq(row, 3*N+t-1) = -1;
    else
        beq(row) = socInit;
    end
end

if socEndMin > socMin
    nUb = 2*N + 1;
else
    nUb = 2*N;
end
Aub = sparse(nUb, nVar);
bUb = zeros(nUb, 1);
for t = 1:N
    Aub(t, 3*N+t) = 1;      bUb(t) = socMax;
    Aub(N+t, 3*N+t) = -1;   bUb(N+t) = -socMin;
end
if socEndMin > socMin
    Aub(nUb, 3*N+N) = -1;
    bUb(nUb) = -socEndMin;
end

lb = zeros(nVar,1);
ub = inf(nVar,1);
ub(N+1:2*N) = pMax;
ub(2*N+1:3*N) = pMax;
lb(3*N+1:4*N) = socMin;
ub(3*N+1:4*N) = socMax;
```

```
options = optimoptions('linprog', 'Display', 'off');
[x, ~, exitflag] = linprog(f, Aub, bUb, Aeq, beq, lb, ub, options);

if exitflag == 1
    pBuy = max(x(1:N), 0);
    pCh = max(x(N+1:2*N), 0);
    pDis = max(x(2*N+1:3*N), 0);
    soc = x(3*N+1:4*N);
else
    pBuy = []; pCh = []; pDis = []; soc = [];
end
end



function pred = predict_load(d, dates, data, method)
    pred = [];
    if strcmp(method, 'A')
        target = d - caldays(7);
        idx = find(dates == target, 1);
        if ~isempty(idx)
            pred = data(idx,:)';
        end
    elseif strcmp(method, 'B')
        season = get_season(d.Month);
        wd = weekday(d);

        seasons = arrayfun(@(x) get_season(x.Month), dates);
        weekdays = weekday(dates);

        mask = (dates < d) & (seasons == season) & (weekdays == wd);
        idx = find(mask);

        if isempty(idx)
            mask = (dates < d) & (weekdays == wd);
            idx = find(mask);
        end

        if ~isempty(idx)
            pred = mean(data(idx,:), 1)';
        end
    end
end
```

```
function pred = predict_pv(d, dates, data, method)
    pred = [];
    if strcmp(method, 'A')
        target = d - caldays(1);
        idx = find(dates == target, 1);
        if ~isempty(idx)
            pred = data(idx,:)';
            pred = set_night_zero(pred);
        end
    elseif strcmp(method, 'B')
        w = [0.30 0.25 0.20 0.15 0.07 0.02 0.01];
        pred = zeros(144,1);
        wSum = 0;
        for k = 1:7
            target = d - caldays(k);
            idx = find(dates == target, 1);
            if ~isempty(idx)
                pred = pred + w(k) * data(idx,:)';
                wSum = wSum + w(k);
            end
        end
        if wSum > 0
            pred = pred / wSum;
            pred = set_night_zero(pred);
        else
            pred = [];
        end
    end
end

function pred = set_night_zero(pred)
    pred(1:29) = 0;
    pred(113:144) = 0;
end

function [emg, costPlan, costEmg, socHist] = simulate_realtime(...
        pBuy, pCh, pDis, loadAct, pvAct, price, dt, ...
        socInit, eta, socMin, socMax, pMax)

    N = length(pBuy);
    emg = zeros(N,1);
    socHist = zeros(N,1);
    soc = socInit;
```

```
    for t = 1:N
        available = pvAct(t) + pBuy(t);
        need = loadAct(t);

        if available >= need
            surplus = available - need;

            chLimP = pMax;
            chLimE   = (socMax - soc) / (eta * dt);
            chNow = min([surplus, chLimP, chLimE]);
            chNow = max(chNow, 0);

            disNow = 0;
            emg(t) = 0;
        else
            deficit = need - available;

            chNow = 0;

            disLimP = pMax;
            disLimE   = (soc - socMin) * eta / dt;
            disNow = min([deficit, disLimP, disLimE]);
            disNow = max(disNow, 0);

            deficit = deficit - disNow;

            emg(t) = max(deficit, 0);
        end

        soc = soc + eta * chNow * dt - disNow * dt / eta;
        soc = max(min(soc, socMax), socMin);
        socHist(t) = soc;
    end

    costPlan = sum(price .* pBuy * dt);
    costEmg  = sum(5 * price .* emg * dt);
end

function write_result2(results, N, dt, filename)
if nargin < 4
    filename = 'result2.xlsx';
end
```

```
nDays = length(results);
if nDays == 0
    warning('没有结果可写入');
    return;
end

timeLabels = cell(1, N);
for t = 1:N
    startMin = (t-1)*10; endMin = t*10;
    sh = floor(startMin/60); sm = mod(startMin,60);
    eh = floor(endMin/60);   em = mod(endMin,60);
    if t == N
        timeLabels{t} = sprintf('%d:%02d-%d:00+1', sh, sm, eh);
    else
        timeLabels{t} = sprintf('%d:%02d-%d:%02d', sh, sm, eh, em);
    end
end

header1 = [{'日期\时间'}, timeLabels(2:end), timeLabels(1), ...
    {'购电总量(kWh)'}, {'购电成本(元)'}];
writecell(header1, filename, 'Sheet', '计划购电量', 'Range', 'A1');

sheet1Rows = cell(nDays, 147);
for i = 1:nDays
    r = results(i);
    sheet1Rows{i,1} = datestr(r.date, 'yyyy-mm-dd');
    buyRotated = [r.pBuy(2:end); r.pBuy(1)];
    for t = 1:N
        sheet1Rows{i, t+1} = round(buyRotated(t), 4);
    end
    sheet1Rows{i,146} = round(sum(r.pBuy) * dt, 4);
    sheet1Rows{i,147} = round(r.dayCostPlan, 4);
end
writecell(sheet1Rows, filename, 'Sheet', '计划购电量', 'Range', 'A2');

header2 = {'日期', '时间段', '充电量', '放电量', '时刻', '储电量'};
writecell(header2, filename, 'Sheet', '充放电量', 'Range', 'A1');

periodRanges = [1 24; 25 48; 49 72; 73 96; 97 120; 121 144];
periodLabels = {'0:00-4:00', '4:00-8:00', '8:00-12:00', ...
    '12:00-16:00', '16:00-20:00', '20:00-24:00'};

nRows2 = nDays * 6;
```

```
sheet2Rows = cell(nRows2, 6);
row = 1;
for i = 1:nDays
    r = results(i);
    dateStr = datestr(r.date, 'yyyy-mm-dd');
    for p = 1:6
        rng = periodRanges(p,1):periodRanges(p,2);
        chVal  = sum(r.pCh(rng))  * dt;
        disVal = sum(r.pDis(rng)) * dt;
        if p == 1
            sheet2Rows{row,1} = dateStr;
        else
            sheet2Rows{row,1} = '';
        end
        sheet2Rows{row,2} = periodLabels{p};
        sheet2Rows{row,3} = round(chVal, 4);
        sheet2Rows{row,4} = round(disVal, 4);
        if p == 1
            sheet2Rows{row,5} = '00:00:00';
            sheet2Rows{row,6} = round(r.socInit, 4);
        elseif p == 2
            sheet2Rows{row,5} = '24:00';
            sheet2Rows{row,6} = round(r.soc(end), 4);
        else
            sheet2Rows{row,5} = '';
            sheet2Rows{row,6} = '';
        end
        row = row + 1;
    end
end
writecell(sheet2Rows, filename, 'Sheet', '充放电量', 'Range', 'A2');

header3 = {'日期', '购电时间段', '购电量'};
writecell(header3, filename, 'Sheet', '紧急购电量', 'Range', 'A1');

nEmgRows = 0;
for i = 1:nDays
    nEmgRows = nEmgRows + sum(results(i).emg > 0.01);
end

if nEmgRows == 0
    sheet3Rows = {'无', '无', 0};
else
    sheet3Rows = cell(nEmgRows, 3);
```

```
        row = 1;
        for i = 1:nDays
            r = results(i);
            dateStr = datestr(r.date, 'yyyy-mm-dd');
            firstOfDay = true;
            for t = 1:N
                if r.emg(t) > 0.01
                    if firstOfDay
                        sheet3Rows{row,1} = dateStr;
                        firstOfDay = false;
                    else
                        sheet3Rows{row,1} = '';
                    end
                    sheet3Rows{row,2} = timeLabels{t};
                    sheet3Rows{row,3} = round(r.emg(t) * dt, 4);
                    row = row + 1;
                end
            end
        end
    end
    writecell(sheet3Rows, filename, 'Sheet', '紧急购电量', 'Range', 'A2');

    fprintf('%s 已保存（%d 天）\n', filename, nDays);
    end

    clear; clc; close all;
    if exist('result4-3.xlsx', 'file'), delete('result4-3.xlsx'); end

    N = 144; dt = 1/6; eta = 0.9;
    soc0 = 6000; socMin = 1200; socMax = 10800; pMax = 5000;
    qSafe = 0.95;
    usePvMargin = true;
    predStart = datetime(2025,2,1);
    predEnd   = datetime(2025,12,31);



    price4Table = readtable('附件 4.xlsx', 'VariableNamingRule', 'preserve');
    price4Dates = dateshift(datetime(price4Table{:,1}), 'start', 'day');
    price4Data  = price4Table{:, 2:145};
    fprintf('实时电价: %d 天 × %d 时段\n', size(price4Data,1), size(price4Data,2));


    loadTable = readtable('附件 2.xlsx', 'Sheet', '小区负载', 'VariableNamingRule',
'preserve');
    loadDates = dateshift(datetime(loadTable{:,1}), 'start', 'day');
```

```
    loadData  = loadTable{:, 2:145};


    pvTable = readtable('附件 2.xlsx', 'Sheet', '光伏发电实际功率', 'VariableNamingRule',
'preserve');
    pvDates = dateshift(datetime(pvTable{:,1}), 'start', 'day');
    pvData  = pvTable{:, 2:145};


    fcRaw = readmatrix('附件 3.xlsx');
    if isempty(fcRaw), error('读取附件 3 失败'); end
    nFcRows = size(fcRaw, 1);
    nFcDays = floor(nFcRows / 4);
    fcData = cell(nFcDays, 1);
    for d0 = 1:nFcDays
        rows = (d0-1)*4 + (1:4);
        fcData{d0} = fcRaw(rows, end-23:end);
    end
    fcBaseDate = datetime(2025,1,1);
    fcDates = (predStart:caldays(1):predEnd)';
    nDays = length(fcDates);



    dayResults = cell(nDays, 1);
    totalCost = 0;
    socNow = soc0;
    nAdjusted = 0;

    diagPlanTotal = 0; diagNeedTotal = 0; diagEmgTotal = 0;
    diagEmgCost = 0; diagPlanCost = 0; diagAdjCost = 0;
    diagNoAdjCost = 0; diagPvBiasSum = 0; diagLdBiasSum = 0;
    diagCnt = 0;


    for i = 1:nDays
        d = fcDates(i);
        iFc = days(d - fcBaseDate) + 1;
        if iFc < 1 || iFc > nFcDays, continue; end
        fc = fcData{iFc};
        if iFc > 1
            prevFc = fcData{iFc-1};
        else
            prevFc = NaN(4, 24);
        end

        idxP = find(price4Dates == d, 1);
        if isempty(idxP)
```

```
    fprintf(' [跳过] %s 附件 4 无电价\n', datestr(d));
    continue;
end
price = price4Data(idxP, :)';

loadPredRaw = predict_load_q3(d, loadDates, loadData);
if isempty(loadPredRaw), continue; end

idxAct = find(loadDates == d, 1);
if ~isempty(idxAct)
    loadAct = loadData(idxAct, :)';
    pvAct   = pvData(idxAct, :)';
else
    loadAct = loadPredRaw;
    pvAct   = staircase_pv_fc(fc(1,:), prevFc(1,:));
end
isLastDay = (d == predEnd);

deltaLd = compute_load_margin_day(d, loadDates, loadData, 'A', qSafe);
if usePvMargin
    deltaPv = compute_pv_bias_q3(d, fcBaseDate, fcData, pvDates, pvData, 30);
else
    deltaPv = zeros(N, 1);
end

pvPlanRaw = staircase_pv_fc(fc(1,:), prevFc(1,:));
pvPlan = max(pvPlanRaw - deltaPv, 0);

nightMask = (pvPlanRaw < 50);
deltaLdUse = deltaLd;
deltaLdUse(~nightMask) = 0;
loadPred = loadPredRaw + deltaLdUse;

[pPlan, ~, ~, ~] = solve_stage0(...
    loadPred, pvPlan, price, N, dt, eta, socNow, ...
    socMin, socMax, pMax, isLastDay);
if isempty(pPlan)
    fprintf(' [优化失败] %s 阶段 0\n', datestr(d));
    continue;
end
pEff = pPlan;

[~,~,~,soc36] = simulate_realtime3(pPlan(1:36), ...
```

```
    loadAct(1:36), pvAct(1:36), price(1:36), 36, dt, eta, ...
    socNow, socMin, socMax, pMax);
socAt36 = soc36(end);

pv6Raw = staircase_pv_fc_stage(fc(2,:), 37);
pv6 = max(pv6Raw - deltaPv(37:N), 0);
nightMask6 = (pv6Raw < 50);
deltaLd6 = deltaLd(37:N);
deltaLd6(~nightMask6) = 0;
loadPred6 = loadPredRaw(37:N) + deltaLd6;

[pAdj6, ~, ~, ~, ~, ~] = solve_stageK(...
    loadPred6, pv6, price(37:N), N-36, dt, eta, socAt36, ...
    socMin, socMax, pMax, pPlan(37:N), isLastDay);
if ~isempty(pAdj6), pEff(37:N) = pAdj6; end

pUpto72 = [pPlan(1:36); pEff(37:72)];
[~,~,~,soc72] = simulate_realtime3(pUpto72, ...
    loadAct(1:72), pvAct(1:72), price(1:72), 72, dt, eta, ...
    socNow, socMin, socMax, pMax);
socAt72 = soc72(end);

pv12Raw = staircase_pv_fc_stage(fc(3,:), 73);
pv12 = max(pv12Raw - deltaPv(73:N), 0);
nightMask12 = (pv12Raw < 50);
deltaLd12 = deltaLd(73:N);
deltaLd12(~nightMask12) = 0;
loadPred12 = loadPredRaw(73:N) + deltaLd12;

[pAdj12, ~, ~, ~, ~, ~] = solve_stageK(...
    loadPred12, pv12, price(73:N), N-72, dt, eta, socAt72, ...
    socMin, socMax, pMax, pPlan(73:N), isLastDay);
if ~isempty(pAdj12), pEff(73:N) = pAdj12; end

pUpto108 = [pPlan(1:36); pEff(37:72); pEff(73:108)];
[~,~,~,soc108] = simulate_realtime3(pUpto108, ...
    loadAct(1:108), pvAct(1:108), price(1:108), 108, dt, eta, ...
    socNow, socMin, socMax, pMax);
socAt108 = soc108(end);

pv18Raw = staircase_pv_fc_stage(fc(4,:), 109);
pv18 = max(pv18Raw - deltaPv(109:N), 0);
nightMask18 = (pv18Raw < 50);
```

```
deltaLd18 = deltaLd(109:N);
deltaLd18(~nightMask18) = 0;
loadPred18 = loadPredRaw(109:N) + deltaLd18;

[pAdj18, ~, ~, ~, ~, ~] = solve_stageK(...
    loadPred18, pv18, price(109:N), N-108, dt, eta, socAt108, ...
    socMin, socMax, pMax, pPlan(109:N), isLastDay);
if ~isempty(pAdj18), pEff(109:N) = pAdj18; end

[emg, pChSim, pDisSim, socTrace, costEmg] = simulate_realtime3(...
    pEff, loadAct, pvAct, price, N, dt, eta, socNow, ...
    socMin, socMax, pMax);

[emgNoAdj, pChNoAdj, pDisNoAdj, socNoAdj, costEmgNoAdj] = ...
    simulate_realtime3(pPlan, loadAct, pvAct, price, N, dt, eta, ...
    socNow, socMin, socMax, pMax);

costPlan = sum(price .* pPlan) * dt;
dP = pEff - pPlan;
costAdjWith = sum(1.5*price.*max(dP,0) + 0.5*price.*max(-dP,0)) * dt;
costTotalWith = costPlan + costAdjWith + costEmg;
costTotalNo   = costPlan + costEmgNoAdj;

if costTotalWith <= costTotalNo
    costAdjFinal  = costAdjWith;
    costEmgFinal  = costEmg;
    pChUse = pChSim; pDisUse = pDisSim;
    socUse  = socTrace; emgUse = emg;
    costTotal = costTotalWith;
    usedAdj = true;
else
    pEff = pPlan;
    costAdjFinal = 0;
    costEmgFinal = costEmgNoAdj;
    pChUse = pChNoAdj; pDisUse = pDisNoAdj;
    socUse  = socNoAdj;  emgUse  = emgNoAdj;
    costTotal = costTotalNo;
    usedAdj = false;
end
totalCost = totalCost + costTotal;
netAdj = pEff - pPlan;
if usedAdj, nAdjusted = nAdjusted + 1; end

dayResults{i}.date = d;
```

```
    dayResults{i}.pPlan = pPlan;
    dayResults{i}.pEff = pEff;
    dayResults{i}.netAdj = netAdj;
    dayResults{i}.pCh = pChUse;
    dayResults{i}.pDis = pDisUse;
    dayResults{i}.emg = emgUse;
    dayResults{i}.socTrace = socUse;
    dayResults{i}.costPlan = costPlan;
    dayResults{i}.costAdj = costAdjFinal;
    dayResults{i}.costEmg = costEmgFinal;
    dayResults{i}.costTotal = costTotal;
    dayResults{i}.socInit = socNow;
    socNow = socUse(end);

    planKwh = sum(pPlan) * dt;
    needKwh = sum(loadAct - pvAct) * dt;
    emgKwh  = sum(emgUse) * dt;
    pvBiasDay = mean(pvPlan(43:102) - pvAct(43:102));
    ldBiasDay = mean(loadPred - loadAct);
    diagPlanTotal = diagPlanTotal + planKwh;
    diagNeedTotal = diagNeedTotal + needKwh;
    diagEmgTotal  = diagEmgTotal + emgKwh;
    diagEmgCost   = diagEmgCost + costEmgFinal;
    diagPlanCost  = diagPlanCost + costPlan;
    diagAdjCost   = diagAdjCost + costAdjFinal;
    diagNoAdjCost = diagNoAdjCost + costEmgNoAdj;
    diagPvBiasSum = diagPvBiasSum + pvBiasDay;
    diagLdBiasSum = diagLdBiasSum + ldBiasDay;
    diagCnt = diagCnt + 1;

    if mod(i,20)==0 || i==1 || i==nDays
        if usedAdj, flagStr = '调整'; else, flagStr = '不调整'; end
        fprintf(' [%d/%d] %s 计划=%.0f 调整=%.0f 紧急=%.0f 总=%.0f SOC=%.1f %s\n', ...
            i, nDays, datestr(d), costPlan, costAdjFinal, ...
            costEmgFinal, costTotal, socNow, flagStr);
    end
end


fprintf('总费用: %.0f 元\n', totalCost);


fprintf('统计天数: %d\n', diagCnt);
```

```
fprintf('方案 A 被选中: %d 天 (%.1f%%)\n', nAdjusted, 100*nAdjusted/max(diagCnt,1));
fprintf('计划购电量总计: %.0f kWh\n', diagPlanTotal);
fprintf('实际需求总计: %.0f kWh\n', diagNeedTotal);
fprintf('差额（计划-需求）: %.0f kWh\n', diagPlanTotal - diagNeedTotal);
fprintf('紧急购电量总计: %.0f kWh (%.2f%%)\n', ...
    diagEmgTotal, 100*diagEmgTotal/max(diagNeedTotal,1));
fprintf('计划费: %.0f 元\n', diagPlanCost);
fprintf('调整费: %.0f 元\n', diagAdjCost);
fprintf('实际总费: %.0f 元\n', totalCost);
fprintf('光伏预报偏差: %.2f kW\n', diagPvBiasSum / max(diagCnt,1));
fprintf('负载预报偏差: %.2f kW\n', diagLdBiasSum / max(diagCnt,1));

write_result3(dayResults, N, dt, 'result4-3.xlsx');
fprintf('result4-3.xlsx 已保存\n');

function margin = compute_load_margin_day(d, dates, data, method, q)

nCol = size(data,2);
idx = find(dates < d);
if numel(idx) < 8
    margin = zeros(nCol,1);
    return;
end

pred = NaN(numel(idx), nCol);
act  = data(idx, :);

seasons = arrayfun(@(x) get_season(x.Month), dates);
weekdays = weekday(dates);

for j = 1:numel(idx)
    k = idx(j);
    if strcmp(method, 'A')
        tgt = k - 7;
        if tgt >= 1 && days(dates(k) - dates(tgt)) == 7
            pred(j,:) = data(tgt,:);
        end
    else
        hist = idx(idx < k);
        m = hist(seasons(hist)==seasons(k) & weekdays(hist)==weekdays(k));
        if isempty(m)
            m = hist(weekdays(hist)==weekdays(k));
        end
```

```
        if ~isempty(m)
            pred(j,:) = mean(data(m,:),1);
        end
    end
end

valid = ~isnan(pred(:,1));
if nnz(valid) < 5
    margin = zeros(nCol,1);
    return;
end
err = max(pred(valid,:) - act(valid,:), 0);
margin = quantile(err, q)';
end



function bias = compute_pv_bias_q3(d, fcBaseDate, fcData, pvDates, pvData, window)

N = 144;
bias = zeros(N, 1);

idx = find(pvDates < d);
if numel(idx) < 8
    return;
end

idx = idx(max(1, end-window+1):end);

pred = NaN(numel(idx), N);
act  = NaN(numel(idx), N);

for j = 1:numel(idx)
    k = idx(j);
    dk = pvDates(k);

    iFc = days(dk - fcBaseDate) + 1;
    if iFc < 1 || iFc > numel(fcData)
        continue;
    end

    if iFc > 1
        prevFc = fcData{iFc-1};
    else
```

```
        prevFc = NaN(4,24);
    end

    fc0 = fcData{iFc};
    pred(j, :) = staircase_pv_fc(fc0(1,:), prevFc(1,:))';
    act(j, :)  = pvData(k, :);
end

valid = ~isnan(pred(:,1)) & ~isnan(act(:,1));
if nnz(valid) < 5
    return;
end

bias = mean(pred(valid,:) - act(valid,:), 1)';
end

function margin = compute_pv_margin_q3(d, fcBaseDate, fcData, pvDates, pvData, q)

N = 144;
margin = zeros(N, 1);

idx = find(pvDates < d);
if numel(idx) < 8
    return;
end

pred = NaN(numel(idx), N);
act  = NaN(numel(idx), N);

for j = 1:numel(idx)
    k = idx(j);
    dk = pvDates(k);

    iFc = days(dk - fcBaseDate) + 1;
    if iFc < 1 || iFc > numel(fcData)
        continue;
    end

    if iFc > 1
        prevFc = fcData{iFc-1};
    else
        prevFc = NaN(4,24);
    end
```

```
    fc0 = fcData{iFc};
    pred(j, :) = staircase_pv_fc(fc0(1,:), prevFc(1,:))';
    act(j, :)  = pvData(k, :);
end

valid = ~isnan(pred(:,1)) & ~isnan(act(:,1));
if nnz(valid) < 5
    return;
end

err = max(pred(valid,:) - act(valid,:), 0);
margin = quantile(err, q)';
end

function s = get_season(monthNum)
    if ismember(monthNum, [3 4 5])
        s = 1;
    elseif ismember(monthNum, [6 7 8])
        s = 2;
    elseif ismember(monthNum, [9 10 11])
        s = 3;
    else
        s = 4;
    end
end

function pred = predict_load_q3(targetDate, histDates, histData)
    N = size(histData, 2);
    refDate = targetDate - days(7);
    idx = find(histDates == refDate, 1);
    if isempty(idx)
        pred = [];
        return;
    end
    pred = histData(idx, :)';
end

function [emg, pCh, pDis, socTrace, costEmg] = simulate_realtime3(...
    pBuy, loadAct, pvAct, price, N, dt, eta, socInit, ...
    socMin, socMax, pMax)

emg       = zeros(N, 1);
```

```
pCh      = zeros(N, 1);
pDis     = zeros(N, 1);
socTrace = zeros(N, 1);
soc = socInit;

for t = 1:N
    supply = pvAct(t) + pBuy(t);
    demand = loadAct(t);

    if supply >= demand
        surplus    = supply - demand;
        chLimE = (socMax - soc) / (eta * dt);
        chNow  = min([surplus, pMax, chLimE]);
        chNow  = max(chNow, 0);

        pCh(t) = chNow;
        soc = soc + chNow * eta * dt;
    else
        deficit     = demand - supply;
        disLimE = (soc - socMin) * eta / dt;
        disNow  = min([deficit, pMax, disLimE]);
        disNow  = max(disNow, 0);

        pDis(t) = disNow;
        soc = soc - disNow * dt / eta;

        remaining = deficit - disNow;
        emg(t) = max(remaining, 0);
    end

    soc = max(min(soc, socMax), socMin);
    socTrace(t) = soc;
end

costEmg = sum(5 * price .* emg) * dt;
end

function [pPlan, pCh, pDis, soc] = solve_stage0(...
    loadPred, pvPred, price, N, dt, eta, socInit, ...
    socMin, socMax, pMax, isLastDay)

nVar = 5 * N;
```

```
f = zeros(nVar, 1);
f(1:N) = price * dt;

nEq = 3 * N;
Aeq = zeros(nEq, nVar);
beq = zeros(nEq, 1);

for t = 1:N
    row = t;
    Aeq(row, t) = 1;
    Aeq(row, N+t) = -1;
    Aeq(row, 2*N+t) = 1;
    Aeq(row, 3*N+t) = -1;
    beq(row) = loadPred(t) - pvPred(t);
end

for t = 1:N
    row = N + t;
    Aeq(row, 2*N+t) = dt / eta;
    Aeq(row, N+t) = -eta * dt;
    Aeq(row, 4*N+t) = 1;
    if t == 1
        beq(row) = socInit;
    else
        Aeq(row, 4*N+t-1) = -1;
    end
end

socReserve = 4500;
Aineq = zeros(1, nVar);
Aineq(5*N) = -1;
bineq = -socReserve;

lb = zeros(nVar, 1);
ub = inf(nVar, 1);
lb(4*N+1 : 5*N) = socMin;
ub(4*N+1 : 5*N) = socMax;
ub(N+1 : 2*N) = pMax;
ub(2*N+1 : 3*N) = pMax;

options = optimoptions('linprog', 'Display', 'off', 'Algorithm', 'dual-simplex');
[x, ~, exitflag] = linprog(f, Aineq, bineq, Aeq, beq, lb, ub, options);
```

```
if exitflag <= 0
    warning('阶段 0 优化未收敛，exitflag = %d', exitflag);
    pPlan = []; pCh = []; pDis = []; soc = [];
    return;
end

pPlan = x(1:N);
pCh = x(N+1 : 2*N);
pDis = x(2*N+1 : 3*N);
soc = x(4*N+1 : 5*N);
end

function [pBuy, pCh, pDis, soc, dPup, dPdown] = solve_stageK(...
    loadPred, pvPred, price, M, dt, eta, socInit, ...
    socMin, socMax, pMax, pPlanSeg, isLastDay)

nVar = 7 * M;

maxReachSoc = socInit;
for t = 1:M
    maxReachSoc = min(maxReachSoc + pMax * eta * dt, socMax);
end
if isLastDay
    socReserveTarget = min(6000, maxReachSoc);
else
    socReserveTarget = [];
end

f = zeros(nVar, 1);
f(5*M+1:6*M) = 1.5 * price(1:M) * dt;
f(6*M+1:7*M) = 0.5 * price(1:M) * dt;

nEq = 3 * M;
Aeq = sparse(nEq, nVar);
beq = zeros(nEq, 1);

for t = 1:M
    row = t;
    Aeq(row, t) = 1;
    Aeq(row, M+t) = -1;
    Aeq(row, 2*M+t) = 1;
    Aeq(row, 3*M+t) = -1;
    beq(row) = loadPred(t) - pvPred(t);
```

```
end

for t = 1:M
    row = M + t;
    Aeq(row, 4*M+t) = 1;
    Aeq(row, M+t) = -eta * dt;
    Aeq(row, 2*M+t) = dt / eta;
    if t > 1
        Aeq(row, 4*M+t-1) = -1;
    else
        beq(row) = socInit;
    end
end

for t = 1:M
    row = 2*M + t;
    Aeq(row, t) = 1;
    Aeq(row, 5*M+t) = -1;
    Aeq(row, 6*M+t) = 1;
    beq(row) = pPlanSeg(t);
end

nUb = 2 * M;
if ~isempty(socReserveTarget), nUb = nUb + 1; end
Aub = sparse(nUb, nVar);
bUb = zeros(nUb, 1);
rowNum = 0;
for t = 1:M
    rowNum = rowNum + 1;
    Aub(rowNum, 4*M+t) = 1;  bUb(rowNum) = socMax;
    rowNum = rowNum + 1;
    Aub(rowNum, 4*M+t) = -1; bUb(rowNum) = -socMin;
end
if ~isempty(socReserveTarget)
    rowNum = rowNum + 1;
    Aub(rowNum, 4*M+M) = -1;
    bUb(rowNum) = -socReserveTarget;
end

lb = zeros(nVar, 1);
ub = inf(nVar, 1);
ub(M+1:2*M) = pMax;
ub(2*M+1:3*M) = pMax;
lb(4*M+1:5*M) = socMin;
```

```
ub(4*M+1:5*M) = socMax;

options = optimoptions('linprog', 'Display', 'off', 'Algorithm', 'dual-simplex');
[x, ~, exitflag] = linprog(f, Aub, bUb, Aeq, beq, lb, ub, options);

if exitflag == 1
    pBuy  = max(x(1:M), 0);
    pCh   = max(x(M+1:2*M), 0);
    pDis  = max(x(2*M+1:3*M), 0);
    soc      = x(4*M+1:5*M);
    dPup = max(x(5*M+1:6*M), 0);
    dPdown = max(x(6*M+1:7*M), 0);
else
    pBuy = []; pCh = []; pDis = []; soc = []; dPup = []; dPdown = [];
end
end

function pv = staircase_pv_fc(fc24, fc24Prev)

N = 144;
pv = zeros(N, 1);

if ~isempty(fc24Prev) && ~isnan(fc24Prev(24))
    pv(1:6) = max(fc24Prev(24), 0);
end

for h = 1:24
    startIdx = (h-1)*6 + 7;
    endIdx   = min(startIdx + 5, N);
    if startIdx <= N
        pv(startIdx:endIdx) = max(fc24(h), 0);
    end
end
end

function pv = staircase_pv_fc_stage(fc24, tStart)

N = 144;
pv = zeros(N - tStart + 1, 1);

for h = 1:24
    absStart = tStart + (h-1)*6;
    for s = 0:5
```

```
        target = absStart + s;
        if target >= tStart && target <= N
            pv(target - tStart + 1) = max(fc24(h), 0);
        end
    end
end
end

function write_result3(dayResults, N, dt, filename)

if nargin < 4
    filename = 'result3.xlsx';
end

valid   = ~cellfun(@isempty, dayResults);
results = dayResults(valid);
nDays  = length(results);

if nDays == 0
    warning('没有结果可写入');
    return;
end

timeLabels = cell(1, N);
for t = 1:N
    startMin = (t-1)*10;
    endMin   = t*10;
    sh = floor(startMin/60); sm = mod(startMin, 60);
    eh = floor(endMin/60);   em = mod(endMin, 60);
    if t == N
        timeLabels{t} = sprintf('%d:%02d-%d:00+1', sh, sm, eh);
    else
        timeLabels{t} = sprintf('%d:%02d-%d:%02d', sh, sm, eh, em);
    end
end

if exist(filename, 'file')
    delete(filename);
end

header1 = cell(1, N + 2);
header1{1} = '日期\时间';
for t = 1:N, header1{t+1} = timeLabels{t}; end
```

```
header1{N+1} = '全天购电量';
header1{N+2} = '全天购电费';
writecell(header1, filename, 'Sheet', '计划购电量', 'Range', 'A1');

sheet1Rows = cell(nDays, N + 2);
for i = 1:nDays
    r = results{i};
    sheet1Rows{i,1} = datestr(r.date, 'yyyy-mm-dd');
    for t = 1:N
        sheet1Rows{i, t+1} = round(r.pPlan(t) * dt, 4);
    end
    sheet1Rows{i,N+1} = round(sum(r.pPlan) * dt, 4);
    sheet1Rows{i,N+2} = round(r.costPlan, 4);
end
writecell(sheet1Rows, filename, 'Sheet', '计划购电量', 'Range', 'A2');

header2 = cell(1, N + 2);
header2{1} = '日期\时间';
for t = 1:N, header2{t+1} = timeLabels{t}; end
header2{N+1} = '全天购电量';
header2{N+2} = '全天购电费';
writecell(header2, filename, 'Sheet', '调整购电量', 'Range', 'A1');

sheet2Rows = cell(nDays, N + 2);
for i = 1:nDays
    r = results{i};
    sheet2Rows{i,1} = datestr(r.date, 'yyyy-mm-dd');
    for t = 1:N
        sheet2Rows{i, t+1} = round(r.netAdj(t) * dt, 4);
    end
    sheet2Rows{i,N+1} = round(sum(r.netAdj) * dt, 4);
    sheet2Rows{i,N+2} = round(r.costAdj, 4);
end
writecell(sheet2Rows, filename, 'Sheet', '调整购电量', 'Range', 'A2');

header3 = {'日期', '时间段', '充电量', '放电量', '时刻', '储电量'};
writecell(header3, filename, 'Sheet', '充放电量', 'Range', 'A1');

periodRanges = [1 24; 25 48; 49 72; 73 96; 97 120; 121 144];
periodLabels = {'0:00-4:00','4:00-8:00','8:00-12:00', ...
                '12:00-16:00','16:00-20:00','20:00-24:00'};
nRows3 = nDays * 6;
sheet3Rows = cell(nRows3, 6);
```

```
row = 1;
for i = 1:nDays
    r = results{i};
    dateStr = datestr(r.date, 'yyyy-mm-dd');
    for p = 1:6
        rng = periodRanges(p,1):periodRanges(p,2);
        chVal  = sum(r.pCh(rng))  * dt;
        disVal = sum(r.pDis(rng)) * dt;

        if p == 1
            sheet3Rows{row,1} = dateStr;
        else
            sheet3Rows{row,1} = '';
        end
        sheet3Rows{row,2} = periodLabels{p};
        sheet3Rows{row,3} = round(chVal, 4);
        sheet3Rows{row,4} = round(disVal, 4);

        if p == 1
            sheet3Rows{row,5} = '00:00:00';
            sheet3Rows{row,6} = round(r.socInit, 4);
        elseif p == 6
            sheet3Rows{row,5} = '24:00';
            sheet3Rows{row,6} = round(r.socTrace(end), 4);
        else
            sheet3Rows{row,5} = '';
            sheet3Rows{row,6} = '';
        end
        row = row + 1;
    end
end
writecell(sheet3Rows, filename, 'Sheet', '充放电量', 'Range', 'A2');

header4 = {'日期', '购电时间段', '购电量'};
writecell(header4, filename, 'Sheet', '紧急购电量', 'Range', 'A1');

nEmgRows = 0;
for i = 1:nDays
    nEmgRows = nEmgRows + sum(results{i}.emg > 0.01);
end
if nEmgRows == 0
    sheet4Rows = {'无', '无', 0};
else
    sheet4Rows = cell(nEmgRows, 3);
```

```
    row = 1;
    for i = 1:nDays
        r = results{i};
        dateStr = datestr(r.date, 'yyyy-mm-dd');
        firstOfDay = true;
        for t = 1:N
            if r.emg(t) > 0.01
                if firstOfDay
                    sheet4Rows{row,1} = dateStr;
                    firstOfDay = false;
                else
                    sheet4Rows{row,1} = '';
                end
                sheet4Rows{row,2} = timeLabels{t};
                sheet4Rows{row,3} = round(r.emg(t) * dt, 4);
                row = row + 1;
            end
        end
    end
end
writecell(sheet4Rows, filename, 'Sheet', '紧急购电量', 'Range', 'A2');

fprintf('%s 已保存（%d 天）\n', filename, nDays);
end
```